Rapport de stage TN09 A23

Exploration d'une solution d'IA générative pour aider au support technique d'une usine logicielle

Réalisé par
TEIXEIRA Alexandre



Dirigé par
Cracco Jean-François et MOUKRIM Aziz

Réalisé à l'Université Technologique de Compiègne
Génie Informatique UTC

Date : 04 Septembre 2023 - 16 Février 2024

1. Remerciements

Au terme de cette expérience professionnelle enrichissante au sein de l'entité *Architecture, Services et Outils* chez **eSNCF**, je tiens à exprimer ma profonde gratitude à plusieurs personnes qui ont joué un rôle clé dans mon intégration et mon évolution au cours de ce stage.

Tout d'abord, mes remerciements les plus sincères vont à **Monsieur Trento**, Directeur de l'entité *Architecture, Services et Outils*. Grâce à sa confiance et son soutien, j'ai eu l'opportunité unique d'intégrer les équipes de l'usine logicielle, un environnement stimulant où j'ai pu me confronter à des défis techniques de haute volée et affiner mes compétences en ingénierie logicielle.

Je souhaite également adresser toute ma reconnaissance à **Monsieur Cracco**, mon maître de stage. Son accompagnement au quotidien, ses conseils avisés et sa disponibilité ont grandement facilité mon intégration et mon apprentissage au sein de l'entreprise. Sa pédagogie et son expertise ont été des atouts précieux dans la réussite de mon stage.

Un remerciement spécial est également dû à **Monsieur Ravaux** et **Madame Mathonnet**, mes suiveurs techniques, pour leur aide précieuse. Leur support technique, leur patience et leur capacité à transmettre leur savoir-faire ont été déterminants dans mon adaptation aux outils et méthodologies propres à l'usine logicielle.

Enfin, je ne saurais clore ces remerciements sans exprimer ma gratitude envers *l'ensemble de l'équipe logicielle*. Leur accueil chaleureux, leur bienveillance et leur volonté de partager leurs connaissances ont grandement enrichi mon expérience. Travailler au quotidien avec des professionnels d'une telle qualité humaine et technique a été une source d'inspiration constante et un facteur déterminant dans mon épanouissement professionnel.

Ce stage a été une étape marquante de mon parcours d'ingénieur, me permettant de lier théorie et pratique dans un contexte d'innovation et d'excellence. Pour tout cela, je vous suis infiniment reconnaissant.

Table des matières

1 Remerciements **i**

2 Résumé technique **1**
2.1 Mots-clés 1

3 Introduction **2**

4 Présentation de l'entreprise et de l'équipe d'accueil **3**
4.1 Introduction 3
4.2 Le Groupe SNCF et e.SNCF solution 3
4.3 L'Usine Logicielle d'e.SNCF 5

5 Présentation du stage **8**
5.1 Le sujet 8
5.1.1 Mise en contexte 8
5.1.2 Réalisation 8
5.2 Les méthodes de travail 9
5.3 Les outils utilisés 10
5.4 Outils de communication 10
5.5 Outils techniques 10
5.6 Prise de recul 11
5.6.1 Intérêt du travail 11
5.6.2 Améliorations à envisager 11
5.6.3 Respect de la vie privée et de l'environnement 11

6 Présentation du domaine de l'intelligence artificielle générative **13**
6.0.1 Machine Learning 13
6.0.2 Deep Learning 13
6.0.3 Intelligence artificielle générative 14

7 Etude sur l'Intelligence Artificielle Générative et Proof of Concept **16**
7.1 Définition de la mission 16
7.1.1 Prendre ses marques 16
7.1.2 Premier pas dans les grands modèles de langage 16
7.1.3 Problématique 18
7.1.4 Autres domaines explorés 19
7.1.5 Conclusion 20
7.2 Réalisation 20
7.2.1 Open source contre Propriétaire 20
7.2.2 Présentation de l'architecture retenue 21
7.2.3 Sélection de la donnée 22
7.2.4 Traitement de la donnée 23
7.2.5 Indexation 24

7.2.6 Recherche vectorielle . . . 25
7.2.7 Développement de la webapp . . . 26
7.2.8 Présentation de la webApp aux équipes . . . 26
7.3 Bilan de ces deux premières missions . . . 27

8 Gestion d'une Urgence Inattendue : Développement du démonstrateur 29
8.1 Introduction . . . 29
8.2 Définition de la mission . . . 29
8.2.1 Contexte . . . 29
8.2.2 Problématique . . . 29
8.2.3 Objectif . . . 29
8.3 Réalisation . . . 30
8.3.1 Enrichissement des métadonnées . . . 31
8.3.2 Parser les documents PDF . . . 31
8.3.3 Mise en place de l'interface utilisateur . . . 33
8.4 Retour . . . 34
8.5 Bilan . . . 34

9 Refonte du PoC et mise en production 35
9.1 Définition de la mission . . . 35
9.1.1 Contexte . . . 35
9.1.2 Problématique . . . 35
9.1.3 Objectif . . . 36
9.2 Réalisation . . . 36
9.2.1 La CI/CD . . . 36
9.2.2 Migration vers Flask . . . 37
9.2.3 Utilisation d'outils . . . 38
9.2.4 Mapper la documentation en graphe . . . 40
9.2.5 ReAct Agent . . . 41
9.2.6 métrologie, traçabilité, audit, facturation . . . 43
9.3 Bilan . . . 44

10 Conclusion 45

11 Bibliographie 47

12 Glossaire 48

A Annexe 49

2. Résumé technique

Durant mon stage de 24 semaines à la Société nationale des chemins de fer français (SNCF) , au sein de l'unité e.SNCF DGA - DEA - ASO - AUTOM - Usine Logicielle (eUL), j'ai été chargé de développer une application web innovante exploitant l'intelligence artificielle générative. Cette application, conçue en Python, a pour objectif de faciliter le processus de support technique en automatisant la récupération de documentation pertinente pour résoudre les incidents liés à l'utilisation des outils CI/CD de l'Usine Logicielle (eUL), tout en fournissant une première solution liée à la fois à la documentation mais aussi aux incidents passés. En se connectant directement au système de ticketing de la SNCF, elle a pour but d'accélérer significativement la résolution des problèmes, améliorant ainsi l'efficacité du support.

La mission comportait également un volet exploratoire crucial, consistant à établir des fondations solides dans le domaine de l'Intelligence Artificielle (IA) générative . Cela impliquait de faire de la veille sur les technologies émergentes et les meilleures pratiques, ainsi que de créer un prototype pour démontrer les capacités et les avantages de l'IA générative dans le contexte professionnel. L'objectif était de fournir à la SNCF une base de connaissances permettant de faciliter le lancement de futurs projets d'IA, en assurant une compréhension approfondie et une application efficace de ces technologies.

2.1. Mots-clés

Intelligence Artificielle générative, Retrieval-Augmented Generation, Agents, Fine-tunning, Application Web, Intégration Continue, Déploiement Continu,

3. Introduction

Au sein du cursus d'ingénierie de l'Université de Technologie de Compiègne, le troisième semestre de la quatrième année (niveau bac+4) est dédié à un stage de six mois en tant qu'assistant ingénieur. Ce stage, effectué au sein d'une organisation du secteur privé ou public, vise à offrir aux étudiants une première expérience professionnelle significative. L'objectif principal est de les immerger dans le monde professionnel pour qu'ils puissent appréhender ses défis et contraintes, tout en mettant en pratique les connaissances acquises à l'UTC, afin d'enrichir leur parcours académique par des apprentissages concrets.

Ainsi, j'ai effectué mon stage au sein de l'Usine Logicielle (eUL) de e.SNCF, une entité proposant de l'intégration continue et du déploiement continu pour l'ensemble des projets du groupe SNCF. Ce stage s'est déroulé à Lyon, dans la Tour Oxygène, pour une durée de 24 semaines.

Ce dossier est un rapport sur l'ensemble de mon travail au sein de l'Usine Logicielle, plus particulièrement sur l'exploration et l'application d'outils utilisant l'intelligence artificielle (IA) générative, les méthodes de travail utilisées, l'intégration dans une équipe, les difficultés que j'ai pu rencontrer et comment j'ai réussi à les surmonter, ainsi que le bilan que je tire de cette expérience.

4. Présentation de l'entreprise et de l'équipe d'accueil

4.1. Introduction

Intégrer l'entité e.SNCF solution, au sein du groupe SNCF (Société Nationale des Chemins de fer Français), représente une formidable opportunité de plonger au cœur d'un environnement technologique avancé et dynamique. Ce stage offre un aperçu privilégié de l'innovation et de l'expertise technique au service de la mobilité et de la logistique digitale. Avant de détailler mon immersion au sein de l'eUL, il convient de situer le contexte en présentant brièvement le groupe SNCF et son entité e.SNCF solution.

4.2. Le Groupe SNCF et e.SNCF solution

La SNCF se distingue comme un pilier majeur du secteur ferroviaire, non seulement au niveau national mais également à l'international. En tant qu'acteur essentiel dans les domaines du transport et de la logistique, le groupe se spécialise dans diverses activités, incluant le transport de passagers et de fret ferroviaire, la gestion des infrastructures ferroviaires, ainsi que le développement de solutions de mobilité durable et innovantes.

"Notre mission est de faciliter vos voyages, d'optimiser le flux de transport pour les personnes et les biens, et d'innover pour la mobilité de l'avenir" (Source : Documentation interne de SNCF).

Pour illustrer la SNCF en quelques chiffres

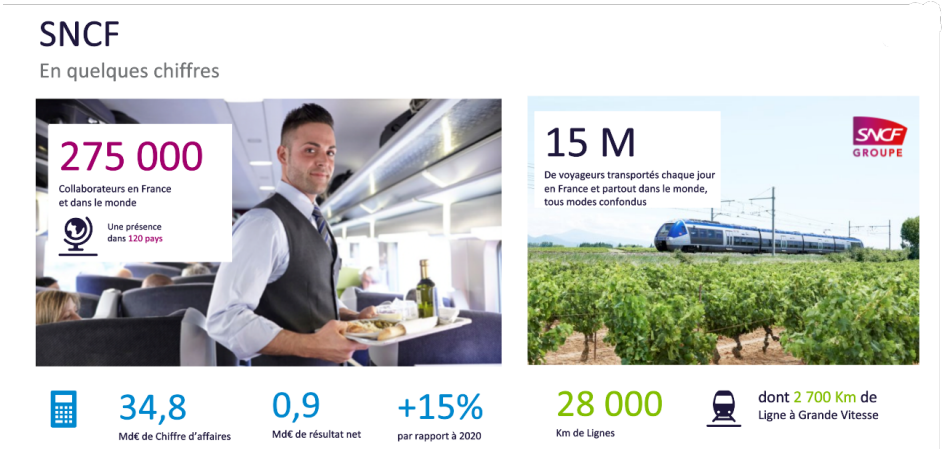


FIGURE 4.1 – Chiffre clés de la SNCF

Pour gérer un tel groupe, la SNCF s'appuie sur une structure organisationnelle bien définie. Cette structure, qui est le fondement même de l'exécution de sa vision, se divise en plusieurs entités spécialisées, chacune jouant un rôle clé dans la réalisation des objectifs du groupe. Pour contextualiser mon rôle, il est nécessaire d'expliquer la structure organisationnelle du groupe SNCF. cac Le groupe se structure autour d'une entité principale, la direction générale du Groupe SNCF, et inclut SNCF Réseau, SNCF Gares et Connexions, Rail Logistics Europe, SNCF Voyageurs, Geodis et Keolis. La direction générale, aussi connue sous le nom de SNCF Holding, coordonne l'ensemble du groupe. Elle héberge également diverses fonctions offrant leurs services à toutes les branches du groupe SNCF. C'est au sein de cette structure, plus précisément dans e.SNCF solution - DGA (- DEA - ASO (architecture service et solution) - AUTOM (automatisation) e Usine Logicielle, que j'ai contribué. (plus d'information sur l'organigramme de l'Usine Logicielle disponible en annexe A.1).



FIGURE 4.2 – graphique de la structure du groupe SNCF

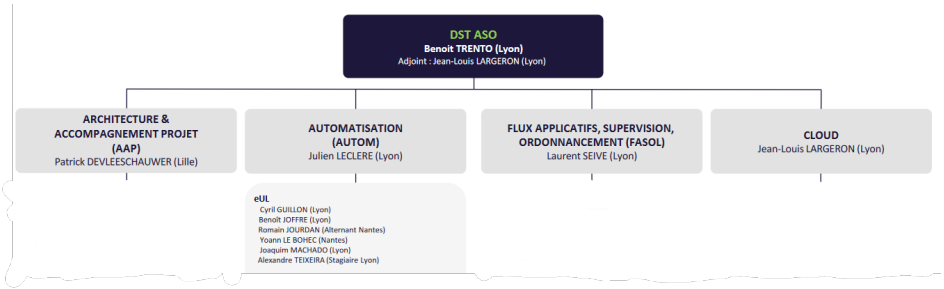


FIGURE 4.3 – graphique de la structure du groupe SNCF

e.SNCF, la branche numérique du groupe, incarne cette volonté d'innovation en concentrant ses efforts sur la digitalisation des services et l'amélioration de l'expérience client à travers des applications et des services en ligne. En s'appuyant sur les dernières technologies, e.SNCF vise à simplifier le parcours client, à optimiser les opérations internes et à explorer de nouvelles voies pour une mobilité plus connectée et accessible. E.SNCF s'engage vers un numérique responsable pour répondre aux enjeux environnementaux et sociaux.

Dans le secteur très concurrentiel du transport et de la mobilité, SNCF et son entité e.SNCF se positionnent solidement grâce à leur expertise technique, leur capacité d'innovation et leur volonté d'offrir des services de qualité. Face à des acteurs internationaux et à l'émergence de nouvelles formes de mobilité, elles continuent de jouer un

rôle prépondérant, en témoigne leur engagement constant en faveur du développement durable et de la transition écologique.

Discuter en détail de toutes les filiales serait trop long, donc je vais plutôt me focaliser sur l'eUL d'AUTOM. Celle-ci est dédiée au développement d'outils pour l'intégration et le déploiement continu à travers tous les projets du groupe SNCF.

4.3. L'Usine Logicielle d'e.SNCF

Qu'est-ce que l'Usine Logicielle ?

L'Usine Logicielle d'e.SNCF (eUL) est une plateforme centrale de livraison continue qui sert l'ensemble des Directions des Systèmes d'Information (DSI) de la SNCF. Son but est de supporter tous les développements e.SNCF, qu'ils soient hébergés sur le système d'information de la SNCF, dans le cloud public ou en local ('on premise').

L'eUL permet aux projets de :

- Stocker leurs référentiels (code source, cas de test, binaires)
- Réaliser l'intégration continue de leur projet sur l'ensemble des technologies de build validé par les architectes
- Déployer en continu son application du développement à la production
- Tester en continu son application (Non Régression, Recette, Exploitabilité, performance et charge)

Ainsi, l'Usine Logicielle est la ligne de service de DevOps dédiée à l'intégration continue, au déploiement continu et aux tests. Elle est au service des DSI et des métiers pour offrir les moyens nécessaires à l'industrialisation des projets informatiques. Elle permet de garder la maîtrise du SI SNCF et contribue à sa sécurité en proposant un socle moderne et modulaire.

L'Usine Logicielle e.SNCF couvre aujourd'hui plus de 1200 projets avec 4500 utilisateurs. Elle est utilisée par 18 DSI avec 80 outils exploités.

On va pouvoir ainsi observer la fabrication des applications dans l'Usine Logicielle.

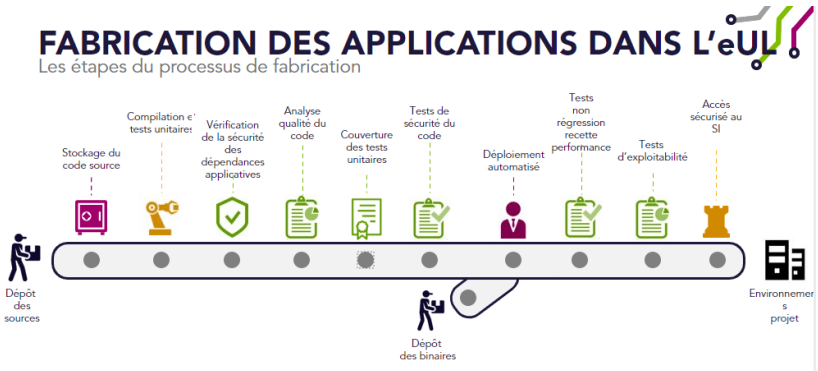


FIGURE 4.4 – fabrication d'une application dans l'eUL

Tous les projets dans l'Usine Logicielle suivent ce cycle de vie et, pour ce faire, il

y a énormément d'outils mis en place pour pouvoir s'adapter à de nombreuses technologies différentes. Pour ce faire, elle a besoin d'une équipe solide capable de maîtriser beaucoup d'outils.

L'usine logicielle c'est qui ?

L'Usine Logicielle est composée de 25 collaborateurs répartis sur 3 sites différents : Lyon, Nantes et Saint-Denis. Ces 25 collaborateurs sont distribués sur 5 grands services qui couvrent l'infrastructure, l'intégration continue (CI), le déploiement continu (CD), une équipe d'IA (IA) et une équipe de support.

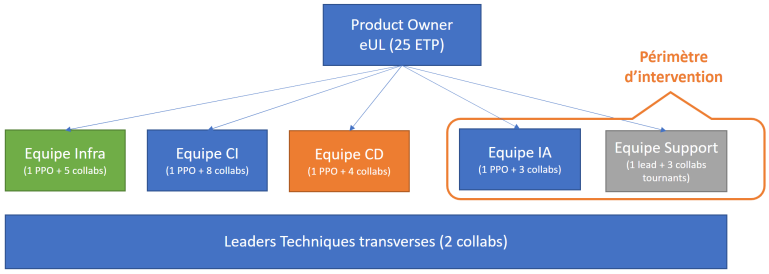


FIGURE 4.5 – organigramme de l'Usine Logicielle

Cette équipe entière permet de déployer des experts dans plus de 40 technologies et outils différents. Comprendre les grandes lignes de la plupart de ces outils fut un vrai défi pour moi tant le domaine est vaste.

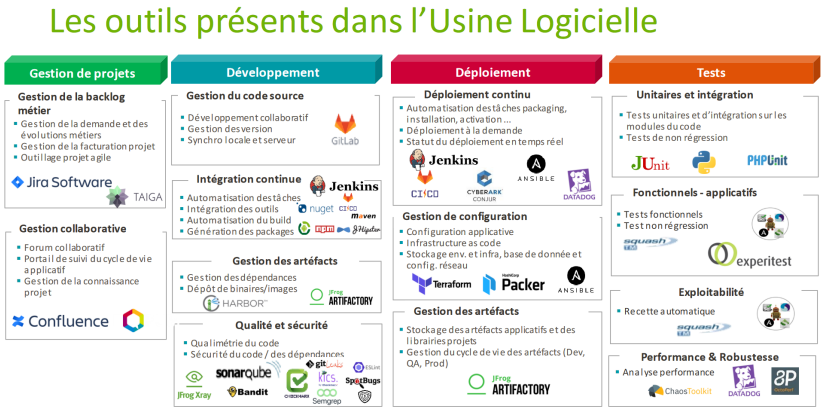


FIGURE 4.6 – Ensemble des outils couvert par l'Usine Logicielle

Je vais dire quelques mots de base sur les services de cette ligne de service tout en détaillant un peu plus mon périmètre d'action.

Équipe Support : Cette équipe, dirigée par Frédéric Souchon, vise à résoudre les incidents rencontrés par les utilisateurs des outils de l'Usine Logicielle. Les incidents peuvent varier, allant de difficultés d'implémentation des outils dans leurs projets à des problèmes plus complexes, tels que des demandes de fonctionnalités spécifiques, qui nécessitent une intervention directe de l'équipe de l'Usine Logicielle. Le processus de gestion des incidents suit un schéma déterminé.

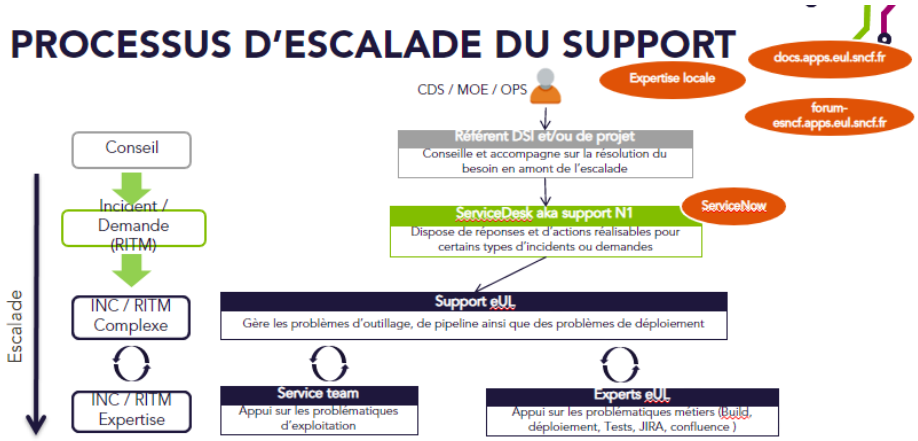


FIGURE 4.7 – vie d'un projet dans l'eUL

Il existe plusieurs niveaux de support, détail important pour la suite de notre rapport. Le niveau N1 est conçu pour adresser les problèmes d'utilisation courante des outils disponibles pour les projets. Généralement, les solutions à ces problèmes se trouvent dans la documentation de l'Usine Logicielle, rendant l'intervention d'un expert inutile. Si le problème est jugé plus critique et complexe, l'incident est alors "escaladé" du niveau N1 vers les experts de l'Usine Logicielle. Nous intervenons dans ce contexte afin de réduire la charge de travail du support et d'accélérer la résolution des incidents.

Equipe IA : L'équipe d'IA est une toute nouvelle entité au sein de la ligne de service, et je suis l'un de ses premiers membres, créée spécifiquement pour ce premier projet en IA générative. Initialement, l'objectif de cette équipe est de réaliser une étude sur les possibilités offertes par l'IA, en l'utilisant notamment pour assister le support de l'Usine Logicielle dans la résolution d'incidents utilisateurs.

Tout au long de ce rapport, je vais vous détailler les actions menées par cette équipe au cours des six derniers mois.

5. Présentation du stage

5.1. Le sujet

Le sujet de mon stage, initialement défini dans la convention de stage, était *Mise en œuvre de l'intelligence artificielle au sein d'ASO*. L'objectif de base visait à permettre à l'eUL de se forger une conviction sur le bon usage des outils de l'IA générative, tout en montant en compétence à travers des premières implémentations internes.

5.1.1 Mise en contexte

L'eUL dispose d'une multitude d'outils à la disposition des utilisateurs pour intégrer la CI/CD et les tests dans leur projet. Malgré une documentation détaillée, ces utilisateurs (qui sont des développeurs, intégrateurs ou chefs de projet) peuvent rencontrer des problèmes techniques liés aux technologies embarquées dans le projet ou à une mauvaise utilisation d'un outil. Pour garantir leur satisfaction, l'eUL accorde une importance majeure à une résolution rapide et efficace de ces incidents. Cependant, en couvrant plus de 1200 projets, ce support nécessite de mobiliser des collaborateurs et donc n'avance pas sur leur feature. Ainsi, grâce à l'IA générative, l'eUL espère réduire la charge de support sur les collaborateurs mais également diminuer les sollicitations des experts techniques de chaque domaine : l'IA permettra de fournir des pistes et des résolutions d'incidents à partir de la documentation et des incidents similaires déjà traités.

5.1.2 Réalisation

Ce défi était de taille, surtout que l'IA générative connaissait un essor significatif. La SNCF commençait tout juste à envisager l'intégration de cette technologie dans ses projets. Tout était à construire. Le projet de stage s'est articulé autour de trois grands axes principaux.

La première fut d'assimiler et comprendre le fonctionnement de l'IA générative, puis implémenter un PoC pour une application web exploitant cette technologie en l'augmentant de la documentation eUL et des incidents similaires résolus, afin d'assister le support technique.

Dans un second temps, ce PoC, ayant démontré son efficacité, m'a permis d'être intégré dans une équipe chargée de présenter le potentiel de l'IA générative aux cadres dirigeants de la SNCF, comme le comité exécutif. Ce qui a mené à un produit nommé SNCF Group-GPT.

Finalement, ma dernière mission a consisté à transformer ce PoC de début de stage en MVP puis à le mettre en production et à en faire un produit clé en main pour les équipes du support. Il était appréciable d'améliorer l'intégration de l'IA au sein

de l'application web, lui permettant d'accéder à certains outils SNCF, en plus de la documentation et des incidents résolus, et de prendre en compte les retours utilisateurs.

5.2. Les méthodes de travail

L'eUL, à l'instar de nombreux projets informatiques actuels, adopte la méthode Scrum, une méthodologie agile.

- **Sprint :** D'une durée de deux à trois semaines, chaque sprint (itération) a des objectifs spécifiques, détaillés dans des cartes Jira (une plateforme permettant la gestion d'outils agile). Ces sprints permettent d'établir des jalons essentiels à la progression du projet.
- **Scrum Meeting :** A la fin de chaque sprint, cette réunion planifie le suivant et se divise en :
  1. *Revue du sprint :* Évaluation des objectifs atteints et non atteints, avec démonstration des travaux achevés.
  2. *Rétrospective du sprint :* Discussion sur l'amélioration continue du travail et des méthodes, avec mise en œuvre des propositions votées dès le sprint suivant.
  3. *Planification du sprint suivant :* Définition des objectifs et tâches à partir du backlog.
- **Backlog :** Inventaire détaillé des tâches nécessaires, régulièrement mis à jour.
- **Statut des tickets :** Progression des tâches selon cinq statuts :
  1. *Selected for development :* Tâche non entamée.
  2. *In progress :* Tâche en cours de réalisation.
  3. *In Study :* Tâche en cours d'étude
  4. *QA :* Tâche complétée, en attente de validation.
  5. *Done :* Tâche achevée et validée.
- **Daily Scrum :** Réunion quotidienne de 15 minutes pour synchroniser l'équipe sur les activités du jour, les défis rencontrés et les plans d'action.

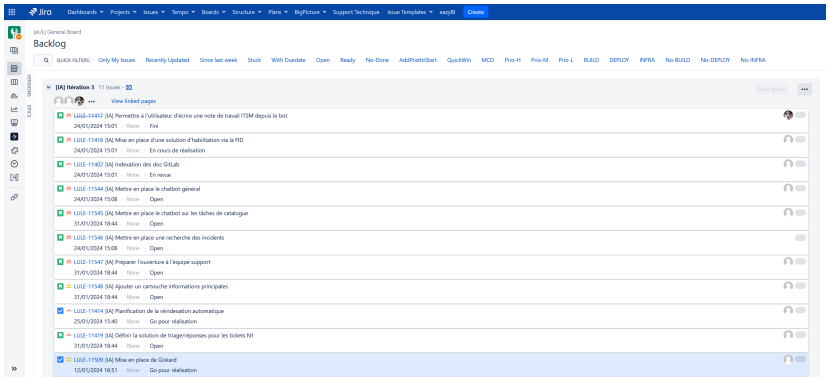


FIGURE 5.1 – Backlog du scrum dans Jira

5.3. Les outils utilisés

5.4. Outils de communication

Les équipes de l'eUL s'appuient fortement sur Microsoft Teams pour leurs communications quotidiennes. Teams est un outil de communication qui permet aux membres de l'équipe de discuter en temps réel, de partager des fichiers et d'organiser des réunions en ligne. Pour la gestion de projets suivant la méthodologie Scrum, l'eUL utilise Jira. Jira est conçu notamment pour le suivi de projets agiles. Il permet aux équipes de créer et de suivre des tâches sous forme de cartes qui évoluent à travers différents statuts, représentant les différentes étapes d'un sprint. Enfin, pour le partage et la gestion de fichiers à l'échelle de l'entreprise, SharePoint de Microsoft est l'outil privilégié. SharePoint permet aux équipes de stocker, organiser, partager et accéder à des informations depuis différents appareils, facilitant ainsi la collaboration et la gestion documentaire.

5.5. Outils techniques

Pour le développement du projet, j'ai eu la liberté de choisir le langage de programmation, à condition qu'il soit supporté par l'eUL. Le choix de Python s'est fait car il est facile d'utilisation et est le langage le plus utilisé pour le développement dans le domaine de l'intelligence artificielle. De plus, avec un unique langage de programmation, j'ai été capable d'interagir avec la solution Cloud Azure, de faire du développement Web et de l'IA générative. Ce choix permet aussi une pérennisation du projet pour trouver des équipes maîtrisant ce langage.

Ce projet nécessite différentes briques qui peuvent être soit on-premise soit dans le cloud. Ainsi, j'ai dû choisir un fournisseur de cloud de la SNCF. Le plus avancé en matière d'IA générative et celui privilégié par l'eUL est Microsoft. J'ai ainsi utilisé Azure pour accéder à toutes les ressources nécessaires au déploiement de l'architecture.

Tout le code a été stocké sur GitLab sur des instances privées, gérées par l'eUL. GitLab est une plateforme de gestion de code source qui utilise Git. Elle permet aux équipes de collaborer sur le code en fournissant un historique détaillé des modifications.

Enfin, j'ai travaillé avec la plateforme de recensement des incidents utilisateur qui est ServiceNow. Cette plateforme est l'outil principal du support eUL, toutes les demandes y sont traitées.

Outils utilisés pour la CI/CD

Ainsi, voici sous forme de liste les outils utilisés pour la CI/CD :

- **Giskar** : Outil de gestion des tests qui permet de définir et de suivre les cas de test pour l'IA générative.
- **Jenkins** : Serveur d'automatisation open source utilisé pour orchestrer le processus de build et d'intégration continue.
- **JFrog Artifactory** : Gestionnaire de dépôts d'artefacts qui sert de registre central pour les binaires et les librairies, facilitant ainsi leur distribution.

— **Ansible** : Outil d'automatisation qui permet de configurer des systèmes, de déployer des logiciels et d'orchestrer des tâches plus complexes.

5.6. Prise de recul

5.6.1 Intérêt du travail

L'intérêt de ces produits finis d'IA générative permet aux utilisateurs d'accéder rapidement à de l'information commentée et de pouvoir interagir avec ces données.

Dans le cadre des démonstrateurs, cela se traduit par la recherche d'informations à travers la vaste base documentaire de la SNCF, Digidoc+, permettant ainsi de trouver des informations précises à travers une multitude de documents. De plus, cela permet à quiconque d'obtenir des explications sur des notions techniques, voire de la vulgarisation prête à l'emploi.

En ce qui concerne le produit destiné au support de l'eUL, il permet, dans un premier temps, d'assister la résolution d'incidents en fournissant des solutions directement issues de la documentation et des incidents résolus. À terme, cet outil pourrait évoluer vers une solution capable de résoudre par elle-même les incidents de niveau 1, supprimant ainsi un niveau de support.

5.6.2 Améliorations à envisager

Les évolutions de ce produit peuvent être multiples. On peut envisager une première itération qui permet d'accéder à davantage d'outils, comme les logs, la récupération de code source dans GitLab, et de proposer des solutions plus pertinentes. Une deuxième évolution serait celle vers une intelligence artificielle multimodale, capable de récupérer plusieurs formes d'information et de la traiter en conséquence, y compris des captures d'écran envoyées par les utilisateurs de l'eUL. Enfin, l'IA générative n'est qu'une étape ; nous tendons vers une IA autonome/action qui, à l'avenir, pourra effectuer directement des tâches comme modifier les codes directement dans GitLab et relancer des pipelines de CI/CD de manière autonome. Parallèlement, comme mentionné précédemment, cette technologie pourrait prendre la place d'un niveau de support en remplaçant le support de niveau 1, dont la résolution est simple et la solution se trouve dans la documentation.

5.6.3 Respect de la vie privée et de l'environnement

L'impact carbone de l'intelligence artificielle générative est considérable et suscite d'importantes préoccupations environnementales, principalement en raison de l'énergie requise pour l'entraînement et le fonctionnement de ces systèmes. Initialement, l'entraînement nécessite 1 000 000 heures de GPU sur des Nvidia A100, avec une consommation de 250W par GPU, soit une consommation totale de 250 000 kWh. Pour les Nvidia H100, la consommation monte à 700W par GPU, équivalant à 700 000 kWh. En comparaison, cela représente la consommation électrique annuelle de 125 à 350 ménages. Si l'on multiplie cela par le nombre de GPU utilisés (les estimations varient de 1 000 à 5 000, mais sont difficiles à vérifier), l'impact devient considérable.

Cependant, cette consommation n'est pas spécifique à la SNCF. Pour évaluer l'empreinte énergétique de l'utilisation d'un tel service, il serait nécessaire de connaître le nombre d'heures d'utilisation de l'IA générative, le nombre d'utilisateurs et le nombre de GPU requis pour exécuter ChatGPT-4 localement pour un nombre donné de personnes. Ces données sont toutefois difficiles à obtenir. Néanmoins, l'importance de l'impact reste significative.

C'est pourquoi, lors de la présentation des différents outils, j'ai mis en place des avertissements sur l'utilisation excessive des IA génératives sous l'angle environnemental.

En ce qui concerne la protection de la vie privée, la SNCF a accès à des environnements privés fournis par Microsoft et OpenAI pour exploiter les différents LLM à disposition. Cela permet de contrôler le cheminement des données envoyées aux LLM afin qu'elles restent en France.

6. Présentation du domaine de l'intelligence artificielle générative

Dans cette partie, nous allons revoir les bases du machine learning, un domaine important de l'intelligence artificielle. Puis, dans un second temps, nous aborderons ce qu'est le deep learning et enfin, nous verrons les particularités de l'intelligence artificielle générative.

6.0.1 Machine Learning

Le machine learning est un domaine de l'intelligence artificielle qui consiste à programmer une machine pour que celle-ci apprenne à réaliser des tâches en étudiant des exemples de ces dernières. D'un point de vue informatique, ces exemples sont représentés par des données que la machine utilise pour développer un modèle. Exemple : $f(x) = ax + bf(x) = ax + b$

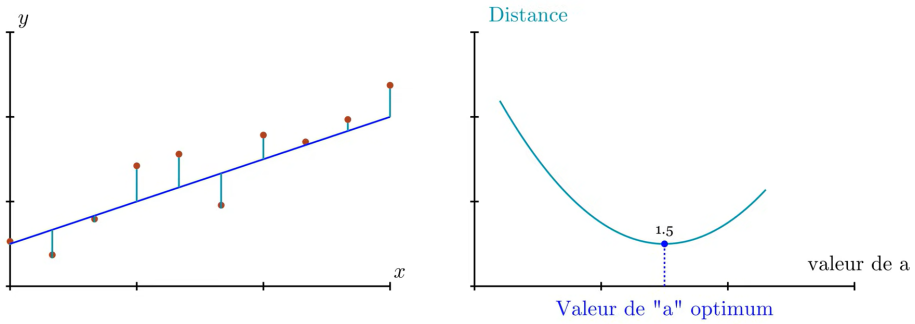


FIGURE 6.1 – fonction d'un modèle basic

L'objectif en machine learning est de trouver les paramètres a et b qui donnent le meilleur modèle possible. C'est-à-dire, le modèle qui s'ajuste le mieux à nos données. Pour ce faire, on programme dans la machine un algorithme d'optimisation qui va tester différentes valeurs de a et b jusqu'à obtenir la combinaison qui minimise la distance entre le modèle et les points de données.

Il existe de nombreux modèles, tels que les modèles linéaires, les arbres de décision, ou encore les machines à vecteurs de support, chacun accompagné de son propre algorithme d'optimisation : la descente de gradient, l'algorithme CART, ou la marge maximale.

Mais alors, qu'en est-il du Deep Learning ?

6.0.2 Deep Learning

Le Deep Learning est un sous-domaine du machine learning dans lequel, au lieu de développer un des modèles mentionnés précédemment, on développe des réseaux de neurones artificiels. Le principe reste le même, c'est-à-dire qu'à partir de données

fournies, la machine utilise un algorithme d'optimisation pour ajuster le modèle à ces données.

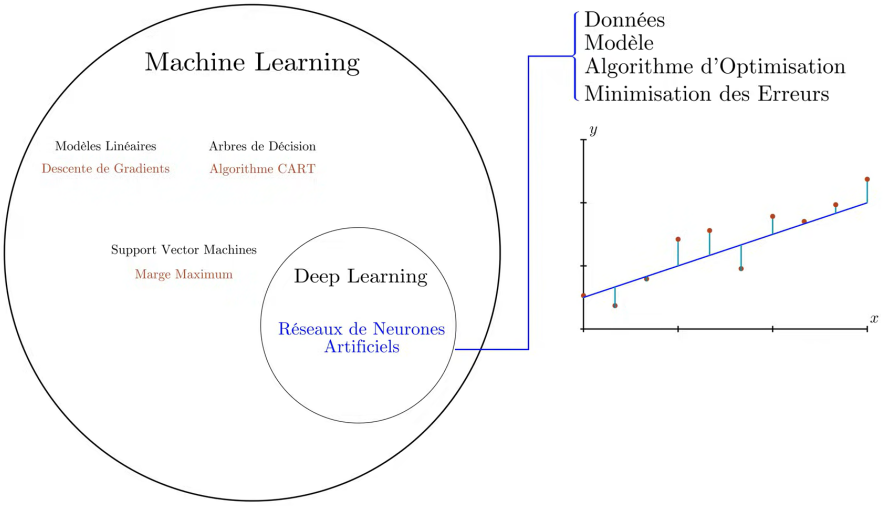


FIGURE 6.2 – deep learning

Cependant, ici, le modèle n'est pas une simple fonction du type $f(x) = ax + bf(x) = ax + b$ mais un réseau de fonctions connectées les unes aux autres, formant un réseau de neurones. Plus ces réseaux sont profonds (plus il y a de couches de neurones), plus la machine est capable de réaliser des tâches complexes, comme reconnaître des objets, identifier une personne...

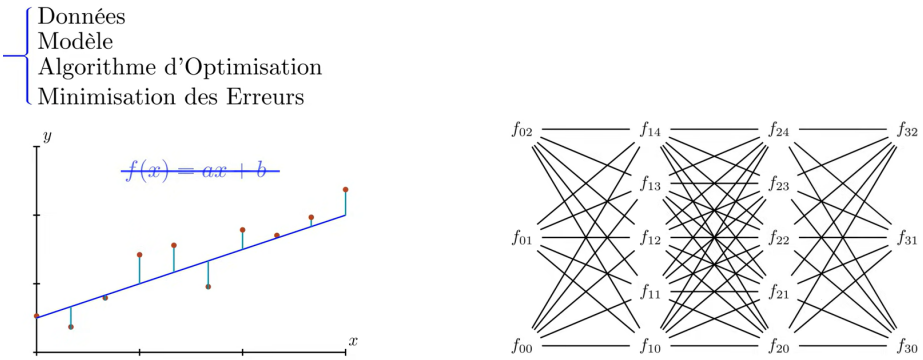


FIGURE 6.3 – réseau de neurones

Maintenant que nous avons vu les bases du deep learning, passons aux modèles d'intelligence artificielle générative

6.0.3 Intelligence artificielle générative

L'intelligence artificielle générative, ou IAg, représente une branche de l'IA Depp learning capable de créer du contenu, comme du texte, des images ou d'autres types de médias, en réponse à des sollicitations spécifiques. Elle devient multimodale lorsqu'elle s'appuie sur divers modèles génératifs ou un modèle unique entraîné avec différents

types de données, lui permettant ainsi de produire une variété de contenus. À titre d'exemple, GPT-4 d'OpenAI peut traiter des entrées textuelles et/ou visuelles.

Cette technologie offre un potentiel d'application quasi universel, bien que le débat sur son équilibre entre risques et avantages soit toujours en cours. L'IAg soulève des préoccupations, ainsi que des défis éthiques, techniques et socio-économiques comparables aux espoirs qu'elle génère. Parmi les risques, on note des risques éthiques (militaire, userpation d'identité...), philosophiques (la conscience, la créativité, la paternité intellectuelle...). Face à une régulation encore insuffisante un premier sommet mondial sur la sécurité de l'IA a été organisé début novembre 2023 à Londres, où l'idée de créer un organisme similaire au GIEC pour informer sur les risques associés à l'IA a été évoquée [2].

Produire un modèle d'IA générative ou Large Language Model (LLM) ou (Grand modèle de langage) est très coûteux, comme nous le verrons dans la prochaine partie **??**. Ainsi, la première approche consiste à comprendre que cette IA est capable de réaliser de nombreuses tâches car elle dispose d'une compréhension générale sur de nombreux domaines différents. Pour interagir avec elle, il suffit de lui "parler". Cette méthode est communément appelée "prompt engineering", et elle permet, à travers des phrases bien formulées, de guider l'IA générative dans son comportement, son rôle, et par exemple, sa zone d'action. Les instructions données à notre modèle d'IA sont plus ou moins bien suivies par les différents modèles, et il reste encore beaucoup à explorer dans le monde du prompt engineering.

Un exemple simple et direct est de lui demander des phrases en emoji, et elle générera sa réponse en emoji.

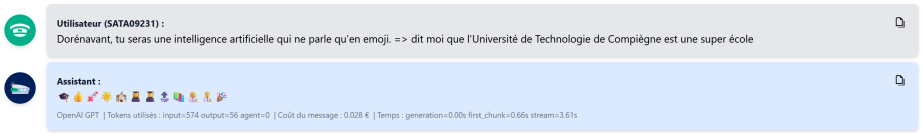


FIGURE 6.4 – prompt

Maintenant que nous avons abordé cette initiation à l'IA générative, nous pouvons commencer à nous pencher sur le véritable sujet du stage.

7. Etude sur l'Intelligence Artificielle Générative et Proof of Concept

7.1. Définition de la mission

7.1.1 Prendre ses marques

Mon premier objectif ne portait pas sur un aspect technique mais humain, il fallait que je me familiarise avec le monde du travail, que j'intègre les équipes et, bien sûr, que j'absorbe toutes les informations relatives au fonctionnement de l'eUL. Face à la nouveauté et à la découverte de l'infrastructure soutenant un tel groupe, j'ai ressenti une certaine appréhension au début. La multitude de nouvelles technologies embarquées nécessitait une formation efficace et rapide sur les outils et techniques utilisés par l'eUL. Ainsi, les premiers jours ont été consacrés à un apprentissage constant de l'utilisation d'Azure, des outils techniques de l'eUL et à la compréhension des problèmes rencontrés par les utilisateurs et des défis auxquels le support technique faisait face dans la résolution d'incidents.

Dans ce contexte, je me suis formé en posant de nombreuses questions à mon équipe. Celle-ci s'est montrée très bienveillante, m'accompagnant jour après jour pour me familiariser avec l'eUL.

Maintenant que j'ai réussi à prendre mes marques, il est temps de s'attaquer au sujet principal du stage : réaliser une première étude sur l'IA générative.

7.1.2 Premier pas dans les grands modèles de langage

Les grands modèles de langage (ou Large Language Models, LLM) sont des modèles d'IA capables d'interpréter et de générer du texte avec une haute pertinence sémantique. Ils permettent ainsi d'analyser des phrases et de générer des réponses extrêmement pertinentes dans un contexte donné. Je ne vais pas détailler toute la technique nécessaire à la création de son propre modèle, car cela constitue un autre sujet et ne rentrerait pas dans le cadre de ce rapport. Mais voici ce qu'il faut savoir.

Le coût

Aborder le coût de création d'un LLM est essentiel, surtout pour les entreprises évaluant l'investissement nécessaire. Les LLMs, comme les modèles Llama 2 de Meta, impliquent des dépenses significatives. Par exemple, pour un modèle de 10 milliards de paramètres, on estime environ 100 000 heures de GPU, tandis qu'un modèle de 100 milliards de paramètres nécessiterait 1 000 000 heures de GPU.

En termes financiers, l'utilisation d'un seul GPU Nvidia A100 (le même type utilisé pour les modèles Llama 2) coûte entre 1 et 2 dollars de l'heure. Cela signifie

qu'un modèle de 10 milliards de paramètres coûterait aux alentours de 150 000 dollars, et un modèle de 100 milliards, environ 1 500 000 dollars, si l'on opte pour une solution de cloud computing commercial.

Il y a également l'option d'acheter les GPUs. Le coût d'achat d'un GPU A100 est d'environ 10 000 dollars, et il en faudrait un millier pour former un cluster, ce qui élève le coût matériel à environ 10 millions de dollars. En y ajoutant les coûts énergétiques, estimés à 100 000 dollars pour un modèle de 100 milliards de paramètres, le total devient considérable.

Ces estimations ne prennent pas en compte le financement d'une équipe qualifiée d'ingénieurs IA, d'ingénieurs de données, et de data scientists, dont le coût peut facilement atteindre 1 million de dollars.

Ces chiffres montrent clairement que développer un LLM depuis zéro représente un investissement majeur, justifiable seulement si le potentiel de retour sur investissement est conséquent et ne peut être atteint par d'autres moyens, comme l'ingénierie de prompts ou l'ajustement de modèles existants.

Comment faire un LLM ?

Un autre obstacle au développement d'un modèle d'IA réside dans la complexité des étapes à suivre. Chaque étape est complexe et nécessite l'expertise de spécialistes pour être menée à bien. Voici les étapes de création :

1. **Traitement des Données** : Cette étape implique la collecte et la préparation des données nécessaires pour entraîner le modèle. Les données doivent être diversifiées, représentatives et de haute qualité pour assurer l'efficacité du modèle.
2. **Architecture du Modèle** : Le choix de l'architecture est crucial pour la performance du LLM. Cette étape définit la structure et les mécanismes fondamentaux qui permettront au modèle de traiter et de générer des langages.
3. **Entraînement à Grande Échelle** : Cette phase concerne la mise en œuvre pratique de l'entraînement du modèle, utilisant des ressources informatiques importantes pour gérer des volumes massifs de données et d'opérations de calcul.
4. **Évaluation** : Après l'entraînement, le modèle est évalué pour vérifier sa précision, sa pertinence et sa performance globale. Cette étape est cruciale pour garantir que le modèle répond aux attentes et aux exigences prévues.

Chaque étape comporte une profondeur technique considérable. Ainsi, j'ai arrêté mes recherches ici pour faire mon modèle et je me suis concentré sur deux options les plus utilisées qui sont le Fine-tuning et le RAG.

Le **Retriever-Augmented Generation, ou RAG**, représente une approche innovante dans le domaine des modèles de langage. Ce concept fusionne deux composantes clés : un système de récupération d'informations (retriever) et un modèle de génération de langage. Cette méthode permet au modèle d'interagir avec de la donnée mise en contexte sans nécessiter un entraînement spécifique sur celle-ci. Ainsi, le RAG permet d'étendre l'utilité des LLMs à nos sources de données.

Le **fine-tuning**, ou ajustement fin, est une technique cruciale dans le domaine de l'apprentissage automatique, en particulier pour les modèles de langage. Cette méthode consiste à prendre un modèle pré-entraîné et à l'ajuster davantage pour une tâche spécifique ou un ensemble de données particulier.

En octobre 2023, les techniques de fine-tuning, n'étant pas jugées suffisamment performantes au regard du coût induit, m'ont conduit à explorer l'utilisation d'une pipeline de RAG pour mon PoC. Cette méthode, bien plus simple à implémenter, semble offrir des performances comparables, voire supérieures, au fine-tuning[6] [4].

Un autre avantage de faire une implémentation de type RAG est que si la donnée évolue régulièrement, le fait de changer la source de données est bien plus simple que de réentraîner un modèle sur cette même donnée. C'est bien plus coûteux de faire du fine-tuning régulièrement que de changer les documents connectés au modèle d'IA.

J'ai éliminé les possibilités que la SNCF a de créer son propre modèle d'IA générative, et j'ai réalisé une étude comparant les performances, les difficultés d'implémentation, ainsi que la cohérence entre le fine-tuning et le RAG. Le choix s'est porté sur une implémentation de type RAG ; je vais donc expliquer les différents problèmes que l'on peut rencontrer.

7.1.3 Problématique

La question essentielle qui se pose est la suivante : *comment réaliser efficacement l'implémentation du RAG pour notre PoC ?* Une certitude se dégage après la lecture de nombreux articles : le RAG est une méthode relativement aisée à mettre en œuvre, mais délicate à peaufiner. Pour réussir une implémentation optimale de ce RAG, il est essentiel de posséder une vision globale des diverses étapes nécessaires à sa réalisation.

Architecture d'un LLM classique et d'un RAG

Voici une architecture de base d'un LLM



FIGURE 7.1 – architecture LLM

L'objectif du RAG est de connecter notre LLM à une base de données vectorielle. En effectuant une recherche à la fois sémantique et vectorielle 7.2.6 dans notre base de données, nous parvenons à extraire les fragments de texte les plus pertinents en lien avec la question de l'utilisateur, et à les fournir en contexte au LLM. Grâce à ces informations supplémentaires et à la question posée par l'utilisateur, notre LLM sera en mesure d'identifier dans les différents extraits la réponse à la question de notre utilisateur et de fournir une réponse adéquate.

Voici à quoi ressemble l'architecture du RAG

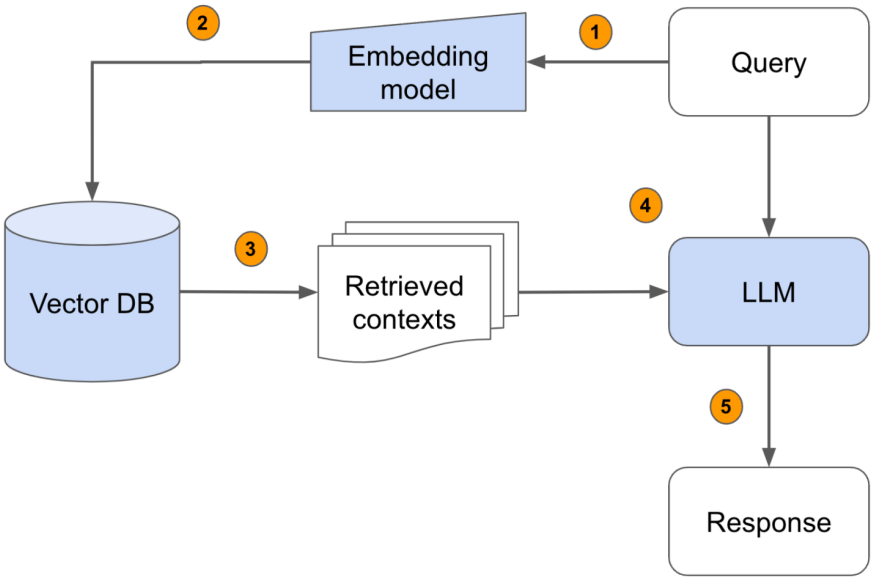


FIGURE 7.2 – architecture RAG

Pour notre base de donnée vectorielle, il faut prendre une ou plusieurs sources de données, à les transformer, les nettoyer et les segmenter en morceaux. Chaque morceau est ensuite converti en vecteur à l'aide d'un modèle d'embedding, stocké dans une base de données vectorielle, puis finalement connecté à notre LLM.

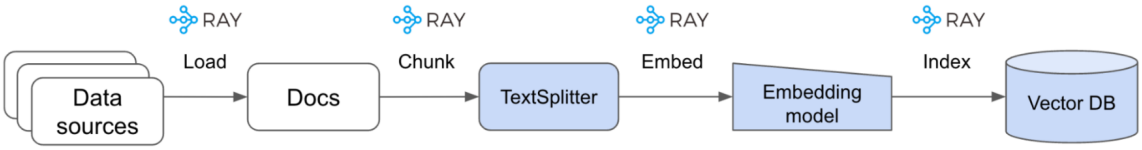


FIGURE 7.3 – traitement de la donnée

Voici l'ensemble des étapes pour faire un RAG :

- Sélectionner une donnée pertinente.
- Traitement et transformer cette donnée pour pouvoir l'exploiter
- Indexer la donnée dans un base de données vectorielle
- Connecter cette base de données avec notre LLM

7.1.4 Autres domaines explorés

Il est évident que le domaine de l'IA générative est extrêmement vaste. Dans ce rapport, il m'est impossible d'exposer toutes les pistes que j'ai pu explorer en ce qui concerne la manière d'implémenter un pipeline RAG connecté à un LLM. Cependant, j'ai établi un graphique représentatif de tous les domaines et sous-domaines que j'ai

explorés durant cette période. Ce graphique illustre l'étendue de mes recherches et la diversité des approches étudiées pour intégrer efficacement les capacités de recherche d'informations à un processus de génération de texte, en vue de créer des réponses plus riches et contextuellement pertinentes.



FIGURE 7.4 – graph des domaines explorés

7.1.5 Conclusion

La première étape du stage est terminée. Je me suis familiarisé avec le monde de l'entreprise, j'ai intégré l'équipe de l'eUL et me suis familiarisé avec l'ensemble des technologies utilisées. J'ai également effectué suffisamment de recherches pour comprendre le fonctionnement des LLM et comment les intégrer dans un projet. La méthode est désormais sélectionnée. La prochaine étape consistera alors à présenter aux équipes une architecture solide pour réaliser mon premier PoC et ensuite le mettre en œuvre.

7.2. Réalisation

7.2.1 Open source contre Propriétaire

Après avoir sélectionné un modèle d'implémentation pour notre IA générative et réalisé de quelques tests pour évaluer la faisabilité, il était essentiel de ne pas se précipiter. En collaboration avec mes responsables techniques, Monsieur Ravaux et Madame Mathonnet, ainsi qu'avec mon manager, Monsieur Leclere, nous avons exploré toutes les options disponibles pour chaque composant de l'implémentation. Nous avons commencé par examiner les solutions open source, tant pour les LLM que pour les bases de données vectorielles, puis propriétaire.

Tout d'abord, l'utilisation de modèles open source contraint l'eUL à devoir opter pour une infrastructure complexe, où le déploiement de ressources GPU dans des services de cloud pour déployer des modèles est nécessaire. De plus, ces modèles semblent moins performants que ceux proposés par Microsoft avec OpenAI.

Du côté des ressources propriétaires, Microsoft, le clouder priviligié par l'eUL, avec sa plateforme Azure, permet un déploiement facile de toutes les briques nécessaires au projet. De plus, ils disposent d'un service Azure OpenAI, qui permet de déployer des LLM GPT et de les requêter via une API. Enfin, la connexion entre ces différents éléments est facilitée par un SDK (Software Development Kit / kit de développement logiciel) Python simple d'utilisation.

Ainsi, la solution propriétaire est bien plus simple à utiliser, à déployer et à maintenir qu'une solution open source. Il faut ajouter à cela que l'eUL reçoit un accompagnement dans le déploiement avec des conseillers à disposition par Microsoft.

7.2.2 Présentation de l'architecture retenue

Maintenant que j'ai identifié les outils et la plateforme sur lesquels je vais déployer mon premier PoC, il est temps de présenter l'architecture de l'application web que je prévois de développer. La première étape consiste à trouver un système de stockage pour mes documents. J'ai utilisé les Azure Blob Storage dédiées au stockage d'objets dans le cloud. Les documents stockés sont ensuite analysés, découpés en segments (ou chunks), nettoyés, et leurs métadonnées, telles que les titres, les liens internes, la date de création, etc., sont extraites. Chaque morceau de texte est alors transformé en vecteur et est stocké dans un index de Azure Cognitive Search avec sa metadonnées associées. Cela offre la possibilité d'effectuer des recherches vectorielles et sémantiques pour identifier les segments de texte les plus pertinents. Ces segments sont par la suite utilisés comme contexte dans notre modèle d'IA générative, lui permettant ainsi de générer des réponses adaptées.

Concernant l'interface utilisateur (frontend) de l'application, il faut considérer que les personnes s'occupant du support reçoivent des tickets d'incidents techniques via la plateforme Service Now. L'infrastructure est alors connectée à cette API. Enfin tout cette mécanique est packagée dans une application web azure.

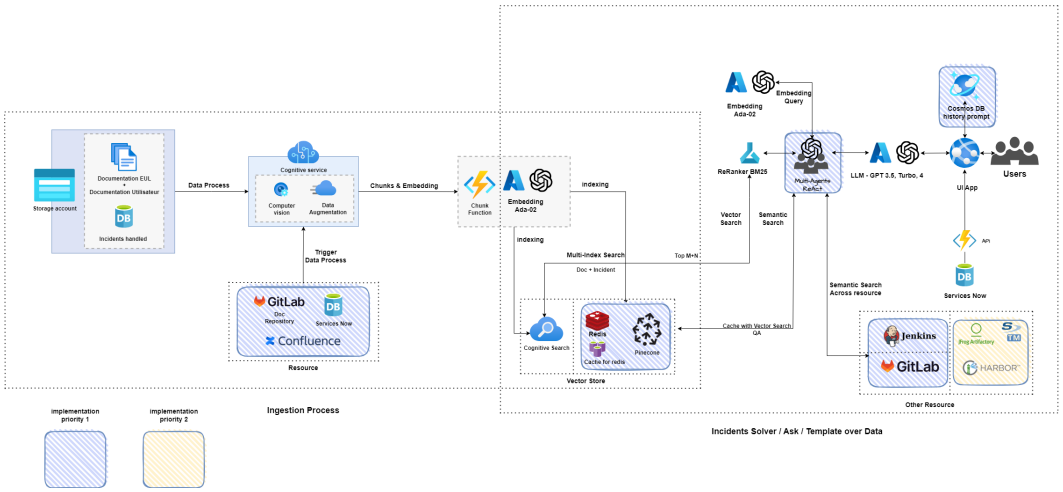


FIGURE 7.5 – Architecture de retenue

Nous avons examiné les grandes lignes de l'implémentation du RAG ainsi que l'architecture de notre application web. Nous pouvons maintenant aborder la première étape, qui est la sélection des données. Dans la prochaine partie, nous allons dresser la liste de toutes les sources de données disponibles, afin de déterminer celles que nous utiliserons ou non.

7.2.3 Sélection de la donnée

Pour résoudre leurs problèmes de manière autonome, sans recourir au support, les utilisateurs peuvent explorer différentes sources de données.

Avant de dresser ce listing, il est important de savoir que toutes les données ne sont pas exploitables. Évidemment, si une donnée est obsolète, dans un mauvais format, ou remplie de bruit, notre IA aura du mal à extraire la réponse pertinente parmi toutes ces informations erronées. Il est important de sélectionner de manière méticuleuse les données que nous allons intégrer dans notre IA. Passons donc au listing.

La première et principale source est la documentation de l'eUL, la plus complète et la plus à jour. Elle se présente sous forme de fichiers Markdown sur GitLab, rédigés et tenus à jour par les membres de la ligne de service. Elle couvre l'essentiel des informations pratiques pour aborder des problèmes simples, souvent classifiés comme des problèmes de niveau N1. Structurée en Markdown avec des règles de formatage claires, cette documentation est précieuse pour sa facilité d'accès à des informations spécifiques comme les titres et les liens. Elle constitue ainsi la ressource la plus significative pour notre usage.

La seconde source d'informations à la disposition des utilisateurs est un forum où les différents projets peuvent échanger sur leurs problématiques et s'entraider. Ce forum regorge d'informations et de solutions souvent très détaillées. Toutefois, le forum vieillit et de nombreuses solutions proposées peuvent être obsolètes. Bien que cette source puisse être pertinente, elle nécessite un important travail de filtrage et ne sera donc pas prise en compte pour le PoC.

La troisième source d'information est le fil Yammer de l'eUL. Il s'agit d'un canal

de diffusion d'informations sur l'activité de l'eUL. C'est une source de données en temps réel qu'il faut considérer. Toutefois, l'implémentation d'un RAG dynamique basé sur des données en temps réel s'avère nettement plus complexe. Par conséquent, nous avons décidé de l'omettre pour le moment.

Enfin, une quatrième source de données, inaccessible aux utilisateurs mais précieuse pour nous, réside dans l'historique des incidents résolus par l'eUL, conservé dans Service Now. Ces incidents, stockés et récupérables via l'API de Service Now, représentent une mine d'informations structurées au format JSON. Bien que leur exploitation nécessite un travail conséquent, ils offrent une donnée structurée et détaillée sur les problématiques rencontrées et résolues par le passé.

Nous avons sélectionné les données que nous allons exploiter avec notre IA. Maintenant, il est nécessaire de traiter ces données, de les transformer pour faciliter au mieux l'extraction d'informations.

7.2.4 Traitement de la donnée

Le traitement de la donnée est une des parties les plus importantes pour rendre notre RAG pipeline efficace. Plus les données sont traitées efficacement, plus les extraits de texte que nous récupérerons dans notre index (voir Section 7.2.5) seront pertinents. Le traitement de la donnée dans notre contexte se déroulera en quatre grandes étapes, qui sont :

1. Prétraitement (voir Section 7.2.4)
2. Analyse (voir Section 7.2.4)
3. Découpage (voir Section 7.2.4)
4. Normalisation (voir Section 7.2.4)

Prétraitement

Le but du prétraitement est de nettoyer les données pour les rendre plus lisibles et compréhensibles pour notre modèle d'IA. Nous allons ainsi éliminer tous les caractères inutiles, redondants ou susceptibles de générer du bruit dans le texte. Les tâches effectuées incluent : la réduction de bruit, les contrôles de cohérence et la déduplication.

Analyse

L'étape d'analyse permet d'extraire toute la métadonnée des fichiers et d'ajouter du contexte au texte qui sera divisé plus tard. Par exemple, pour un format markdown (texte balisé), il est très simple d'extraire tous les titres et les liens dans un fichier.

Découpage

Le découpage, ou "chunking" en anglais, vise à diviser les fichiers Markdown en segments afin de fournir ces extraits comme contexte à notre LLM. Il existe diverses méthodes de découpage, qui varient selon le texte. Une des meilleures pratiques pour le découpage est de procéder de manière récursive, afin de segmenter le texte aux

emplacements les plus appropriés, tels que la fin d'une phrase, d'un paragraphe, ou d'un bloc de code. De plus, pour enrichir le contexte, chaque segment de texte est juxtaposé avec le précédent et le suivant, offrant ainsi une perspective plus large pour chaque extrait. Il est important de noter que la longueur des segments influence directement la quantité de contexte disponible pour l'IA : plus un segment est long, plus il contient de contexte, mais sa complexité à être traité augmente également, rendant la recherche de réponses spécifiques plus ardue. Inversement, des segments plus courts simplifient la localisation d'informations précises mais offrent moins de contexte à l'IA.

Normalisation

La normalisation des données consiste à standardiser la manière dont elles seront présentées à notre IA générative, simplifiant ainsi la restitution du contexte. Cette normalisation s'applique non seulement au texte mais aussi aux données numériques, aux dates, et à d'autres éléments non textuels. Elle assure une homogénéité des données, de sorte que l'IA ne privilégie pas un segment de texte par rapport à un autre, garantissant une plus grande constance dans la restitution des informations recherchées.

Résultat

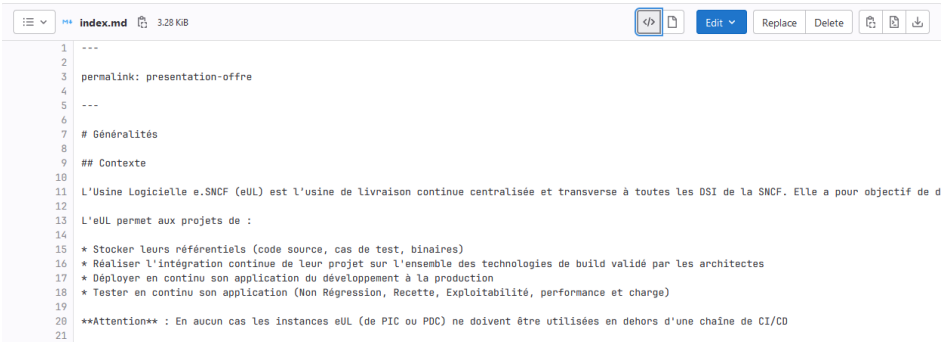


FIGURE 7.6 – donnée avant traitement

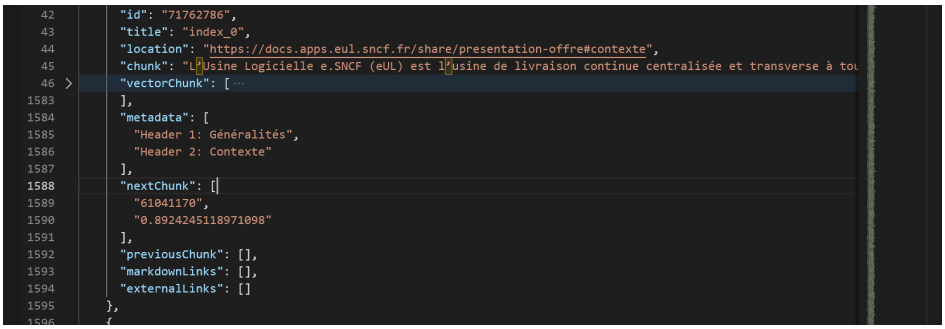


FIGURE 7.7 – donnée après traitement

7.2.5 Indexation

La phase d'indexation consiste à stocker les données transformées au sein d'un index et à convertir le contenu textuel d'un fragment de texte en représentation vecto-

rielle. Cette partie, nommée vectorChunk, est dissimulée car trop longue dans l'illustration référencée 7.7. Ce processus est désigné sous le terme d'« embedding », ou Plongement lexical en français. Cette technique offre une représentation vectorielle des mots ou des phrases, permettant à chaque phrase d'avoir sa propre représentation vectorielle. Les phrases présentant des similarités semblent plus proches les unes des autres dans cet espace vectoriel. Le modèle d'embedding utilisé est un modèle nommé ADA par OpenAI et est un modèle d'espace vectoriel d'une dimension de 1536. Il est important de savoir que ce modèle peut être également fine-tuné pour vectoriser des phrases avec des termes techniques spécifiques plus efficacement.

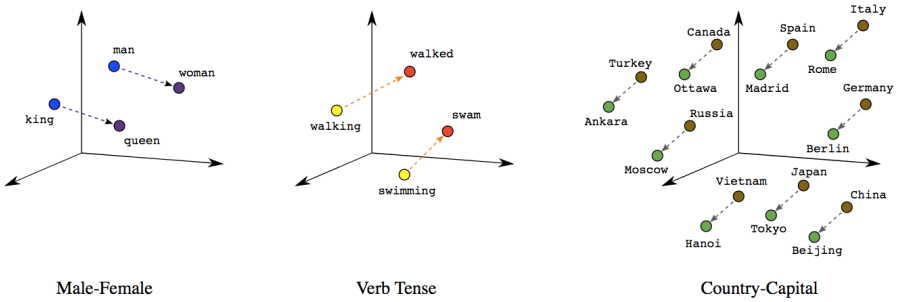


FIGURE 7.8 – représentation vectorielle de mots

7.2.6 Recherche vectorielle

La méthode la plus couramment employée pour cette recherche vectorielle est la Similarité cosinus, qui calcule le cosinus de l'angle entre deux vecteurs selon la formule suivante :

$$\cos\theta = \frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$

La valeur du cosinus, donc celle calculée ici pour $\cos\theta$, se situe dans l'intervalle **[-1,1]**. Une valeur de -1 signifie que les vecteurs sont opposés, une valeur de 0 indique des vecteurs indépendants (orthogonaux), et une valeur de 1 des vecteurs colinéaires avec un coefficient positif. Les valeurs intermédiaires permettent d'évaluer le degré de similarité.

Dans notre contexte, la représentation vectorielle de la question de l'utilisateur est comparée à l'ensemble du texte, et les phrases dont les résultats sont les plus proches de 1 sont sélectionnées.

Cependant, la réalité est plus complexe, puisque nous effectuons une recherche hybride dans cet index. Avec Azure Cognitive Search, les champs vectoriels peuvent coexister avec des champs textuels et numériques, ce qui autorise la formulation de recherche à la fois vectorielle et sémantique exécutées en parallèle. Cette technique permet une restitution des morceaux de texte les plus pertinents de façon beaucoup plus efficace.

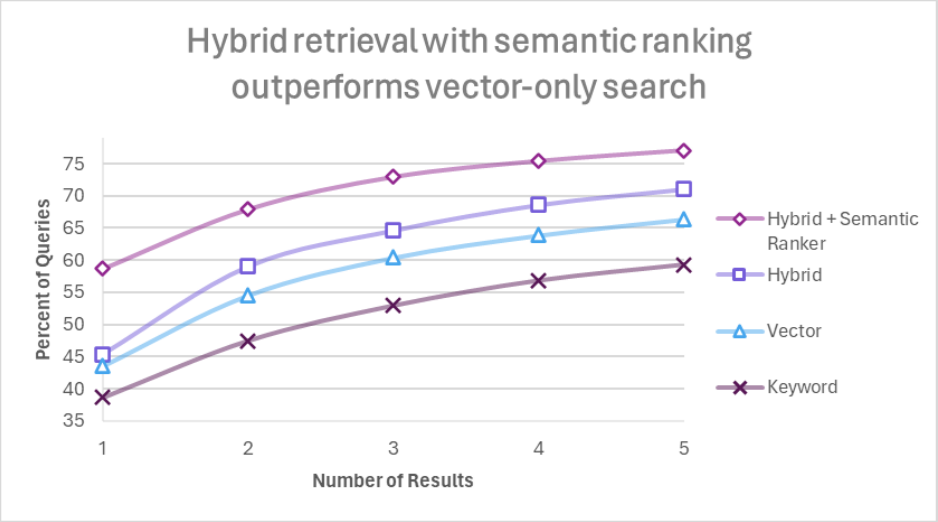


FIGURE 7.9 – Comparaison entre recherche Mots-clés / Vectorielle / Hybride / Hybride + semantic + Ranker

Maintenant que nous avons toutes nos briques de backend pour notre application, nous allons développer le frontend, la partie avec laquelle les utilisateurs vont interagir et pouvoir dialoguer avec notre modèle d'IA.

7.2.7 Développement de la webapp

Lors du développement de la webapp, j'ai dû opter pour quelque chose de simple, rapide, et efficace pour le PoC, car l'objectif n'était clairement pas de concevoir une application web esthétique. N'étant pas développeur web, pour cette première version, je me suis orienté vers un framework émergent qui commence à faire ses preuves dans le monde des data scientists et de l'IA. Ce framework est Streamlit. Il permet, de manière efficace, rapide et dynamique, d'accéder à une multitude de modules créés par la communauté et de les intégrer selon nos besoins dans une page web. Il facilite également les interactions dynamiques avec ces modules.

7.2.8 Présentation de la webApp aux équipes

Il a fallu établir une première interface homme-machine (IHM) initiale pour que le personnel de support puisse s'y retrouver facilement. J'ai ainsi utilisé Figma, un éditeur d'interface utilisateur, pour concevoir une première représentation de cette application web, lui ai attribué un nom, puis l'ai développée en Python avec Streamlit. L'interface est simple mais efficace. Sur la droite, on trouve une liste déroulante avec tous les incidents actuellement ouverts sur ServiceNow et un indicateur de température qui permet de jauger la créativité de l'IA. Au milieu à gauche, nous avons, dans l'ordre : le numéro de l'incident, sa description, les informations relatives à l'incident extraites par l'IA, et les sources qu'elle a utilisées pour répondre à la question (donc à l'aide d'une partie de la documentation et des incidents similaires résolus). Sur la droite, on trouve la première réponse de l'IA générative avec différentes étapes qui pourraient permettre de résoudre l'incident. Enfin, en bas, il y a la possibilité de continuer la discussion avec l'IA pour pouvoir la guider dans ses réponses.

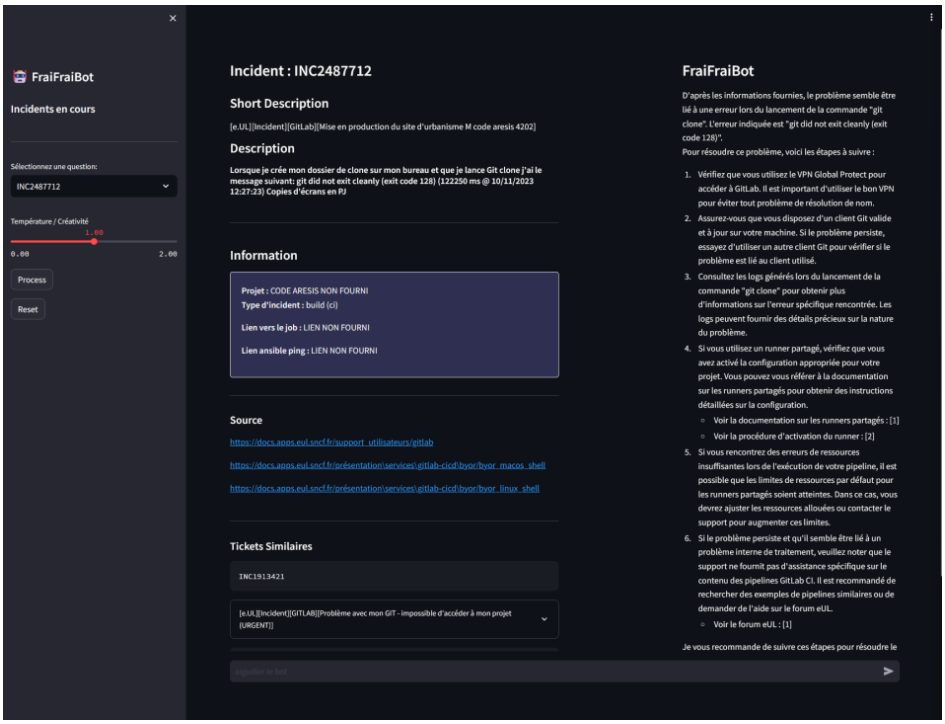


FIGURE 7.10 – application web du PoC

Cette première présentation à l'équipe fut un franc succès et tout le monde a apprécié le travail accompli. L'application a d'ailleurs suscité beaucoup d'intérêt, surpassant les attentes initiales au-delà de l'eUL, ce qui a déclenché la proposition de ma troisième mission, comme décrit dans la section référencée 8. Elle s'est déroulé après presque trois mois de stage le jeudi 16 novembre. C'était le premier jalon très important de ce stage avec des délais un peu serrés. Mais j'étais assez fier du travail présenté.

7.3. Bilan de ces deux premières missions

Le bilan de ces deux premières missions est plutôt satisfaisant pour un premier pas dans le monde de l'entreprise. Le travail réalisé a consisté en un état de l'art des technologies actuelles de l'IA générative. Cela a inclus le fait de lister la grande majorité des façons d'implémenter celles-ci dans une application web. Une étude a été menée pour faire un comparatif qui lie coût, performance et complexité d'intégration afin de pouvoir choisir une direction sur laquelle se lancer. Après cela, nous avons dû choisir une architecture de système en comparant les modèles open source et propriétaires, les serveurs d'hébergement de l'application web, ainsi que les bases de données qui allaient stocker les données. Ainsi, nous avons choisi le cloud de Microsoft qui, avec sa plateforme Azure, permet d'accéder facilement à l'ensemble des ressources nécessaires pour réaliser ce PoC. Puis est venue la réalisation ; il a fallu, en très peu de temps, appliquer tous les concepts fraîchement appris et créer une architecture de toutes pièces. Enfin , il y a eu la présentation de l'application web aux équipes de l'eUL.

Cette application web n'est pas parfaite, et de nombreux aspects peuvent être améliorés (nous verrons cette partie dans la section 9). Toutes ces imperfections sont

normales pour un PoC. C'est une des choses les plus importantes que j'ai apprise lors de ces premières missions. Il ne fallait pas que ce PoC soit parfait, avec toutes les implémentations réalisées de façon irréprochable, comme un traitement des données parfait, l'utilisation de modèles de LLM au millimètre près avec des instructions soignées, ou encore une logique complexe pour rendre l'IA plus intelligente. Il fallait un produit de test, montrer une application parmi une multitude de possibilités de l'IA. Durant ces missions, j'ai parfois eu du mal à terminer certaines tâches car je voulais les optimiser parfaitement, mais cela n'est pas conforme à la méthode agile qu'ils utilisent à l'eUL. Il fallait une première itération, puis l'améliorer au fur et à mesure.

Ce que je retiens de cette mission est que tout d'abord j'ai bien assimilé le fait de faire des itérations incomplètes et de les améliorer lors de chaque itération suivante. De plus, ces deux premières missions m'ont donné confiance en mon travail et j'ai réussi à prouver à l'entreprise que j'étais capable d'assimiler une forte charge de travail, d'apprendre et d'appliquer des concepts rapidement. L'appréhension que j'avais au début de mon stage s'est peu à peu effacée au fur et à mesure que les tickets Jira passaient et que les tâches étaient accomplies. Enfin, je suis très largement monté en compétence sur le code produit qui, au début, était brouillon, mais à force de travailler avec des gens ultra compétents dans leur domaine, on améliore énormément la qualité de notre code pour qu'il soit relu, compris facilement et qu'il soit performant.

8. Gestion d'une Urgence Inattendue : Développement du démonstrateur

8.1. Introduction

Le lendemain de la présentation et de la conclusion des deux premières missions, j'ai été intégré dans une équipe pour la réalisation d'un démonstrateur d'IA générative destiné aux cadres dirigeants. Cette mission revêtait une importance capitale pour e.SNCF, désireuse de mettre en avant ses compétences et son savoir-faire en matière d'innovation technologique. Ainsi, la mission a été confiée à l'eUL au dernier moment et sur la base de l'infrastructure développée pour le PoC, une équipe fut constituée afin de créer un démonstrateur à l'intention des cadres dirigeants.

8.2. Définition de la mission

8.2.1 Contexte

L'enjeu principal résidait dans la nécessité de proposer un démonstrateur sécurisé, utilisable par les cadres dirigeants montrant les capacités de l'IA générative. Dans le cadre d'une rétrospective sur les projets innovants, e.SNCF se voit contrainte de présenter, en moins d'une semaine, les dernières innovations en matière de numérique et de nouvelles technologies. Fort de la réussite de ma présentation, mes supérieurs ont décidé de confier la tâche de développer un démonstrateur sur l'IA destiné aux cadres dirigeants à l'eUL. Le délai imparti était de quatre jours, et l'enjeu, considérable. Si nos connaissances de base étaient solides, il nous fallait néanmoins adapter les technologies existantes à de nouvelles données et recourir à des solutions plus matures pour la réalisation de cette application web.

8.2.2 Problématique

La problématique principale de ce projet est le temps imparti qui est très court. Redéployer toute la structure faite pour le PoC et l'implémenter de façon plus mature dans un contexte d'urgence est un défi de taille.

8.2.3 Objectif

L'objectif, bien qu'ambitieux, était clairement défini : concevoir et déployer une application web démontrant les capacités de l'IA générative. Cette application devait :

— Intégrer Flask en backend pour une connexion sécurisée conforme aux standards de la FID de la SNCF.

— Permettre la connexion à divers modèles d'IA générative, avec la possibilité de basculer dynamiquement entre eux.
— Assurer l'interaction de tous les modèles avec une base documentaire générique de la SNCF.
— Supporter un volume élevé d'utilisateurs simultanés sur l'utilisation des modèles d'IA générative déployés.
— Offrir une interface utilisateur soignée pour la présentation.

Ces objectifs définissaient le cadre du projet. Mon rôle spécifique consistait à partager mon expérience sur le fonctionnement et l'implémentation de l'IA générative et d'indexer la donnée sélectionnée.

L'équipe projet était composée de trois développeurs : Charlie Mathonnet, Guillaume Martinez et moi-même, sous la direction de monsieur Ravaux.

8.3. Réalisation

Pour mener à bien ce projet, l'équipe et moi-même nous sommes répartis la charge de travail principale. Les étapes importantes pour cette application étaient : le traitement et la vectorisation des données dans un index d'Azure Cognitive Search, l'implémentation de l'algorithme utilisant le modèle d'IA avec connexion à la base de données, la création d'un load balancing (équilibrage de charge) entre les différentes instances du modèle pour éviter la surcharge des modèles déployés et enfin packager tout cela dans une application web avec une connexion sécurisée.

Ainsi, Guillaume Martinez et moi nous étions chargés de l'implémentation de l'utilisation du modèle d'IA, du développement du front-end et du back-end de l'application et du traitement des données. Charlie Mathonnet a été chargé de l'équilibrage de charge.

"Dans la suite de ce chapitre, je ne vais pas aborder la partie concernant l'utilisation des LLM, car cela a été traité dans le chapitre précédent (voir section **??**). En revanche, je me concentrerai sur le traitement de données non structurées, telles que les fichiers PDF.

Division du traitement des données en sous-tâches

Face à l'objectif de réaliser cette tâche en seulement quatre jours, l'efficacité était primordiale. Les étapes pour les trois jours à venir étaient donc :

— Récupérer le corpus documentaire et collecter un maximum de métadonnées stockées dans une plateforme nommée Digidoc+.
— Identifier, à travers une série de tests, le meilleur outil de parsing de PDF en établissant une banque de tests.
— Segmenter chaque document et évaluer les performances en fonction de la taille des segments de texte.
— Créer un index dans Azure Cognitive Search en intégrant un maximum de métadonnées et vectoriser tous les documents au sein de cet index.

8.3.1 Enrichissement des métadonnées

Après avoir consulté les experts métier et récupéré le corpus documentaire, il était nécessaire de trouver une méthode pour enrichir les métadonnées de chaque document. Sans accès à l'API de Digidoc+ pour extraire directement les métadonnées, j'ai dû concevoir un script à exécuter dans mon navigateur, lequel effectuait une série de requêtes en boucle pour télécharger les fichiers CSV associés à chaque document et contenant des metadonnées comme le sujet du document, la taille...

8.3.2 Parser les documents PDF

Les documents PDF représentent des données non structurées qui peuvent adopter une multitude de formes. Récupérer les informations à travers un PDF est alors compliqué. Cependant, c'est l'un des formats les plus utilisés pour stocker et partager de l'information et la plupart des documents du groupe sont au format PDF. Je devais alors trouver le moyen le plus efficace de parser les PDF avec des outils soit propriétaires, soit open source.

Pour bien comprendre le format des documents texte, d'un côté, nous avons les documents structurés comme les documents Word, HTML ou Markdown, qui contiennent des balises, et de l'autre, nous avons les documents non structurés comme les PDF ou les scans, qui ne sont pas directement lisibles par une machine [5].

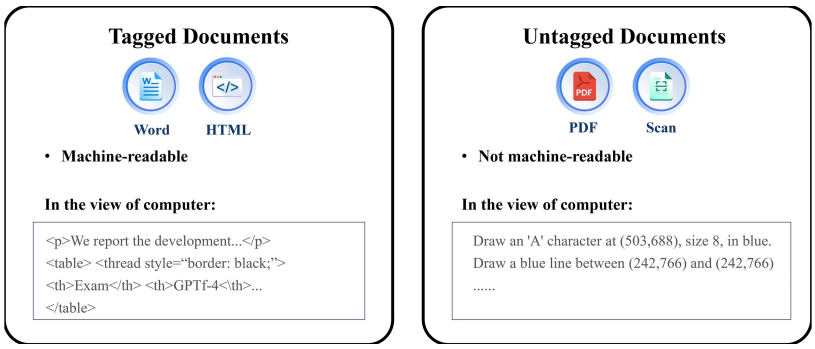


FIGURE 8.1 – Document structuré et non structuré

Tout d'abord listons les problèmes rencontrés lorsqu'on essaie de parser un PDF.

Complexité de mise en page

Les PDF peuvent contenir des mises en page complexes et une multitude de formes, telles que du texte sur deux ou trois colonnes, des tableaux, des images, et des tableaux sous forme d'image. Cette diversité de mises en page rend l'extraction de données structurées extrêmement compliquée.

Problème d'encodage des polices

Les fichiers PDF utilisent divers systèmes d'encodage de polices, et certains de ces systèmes ne correspondent pas directement à l'Unicode. Ainsi, l'extraction de texte est compliquée et des caractères indésirables peuvent s'immiscer, ce qu'on appelle du bruit dans l'extraction de texte.

Stockage de texte non linéaire

Les PDF ne stockent pas le texte dans l'ordre dans lequel il apparaît sur la page. Au lieu de cela, ils stockent le texte dans des objets pouvant être placés n'importe où sur la page. Cela signifie que l'ordre du texte dans le code peut ne pas correspondre à l'ordre du texte tel qu'il apparaît visuellement.

Outils disponibles

Il y a bien d'autres facteurs qui font que les PDF posent un vrai problème pour extraire de l'information. Ainsi, des outils ont été développés pour essayer de parser au mieux les PDF. Il y en a des open source et des propriétaires. Le plus connu, en python et en open source est PyPDF2, une librairie Python qui permet de parser des PDF de façon assez simple, et au niveau propriétaire, les deux qui fonctionnent plutôt bien sont les parseurs de Microsoft [1] et d'Amazon [3].

Aujourd'hui, il existe deux grands type d'architectures pour analyser les PDF. La première architecture est basée sur des règles, comme ce que fait PyPDF2. L'autre méthode est une approche via l'IA avec un modèle d'apprentissage profond, ce que proposent Microsoft et Amazon avec leurs outils.

PyPDF2

Voici une application d'extraction d'un PDF comprenant à la fois des tableaux, du texte en longueur, et une partie de texte séparée en deux colonnes.

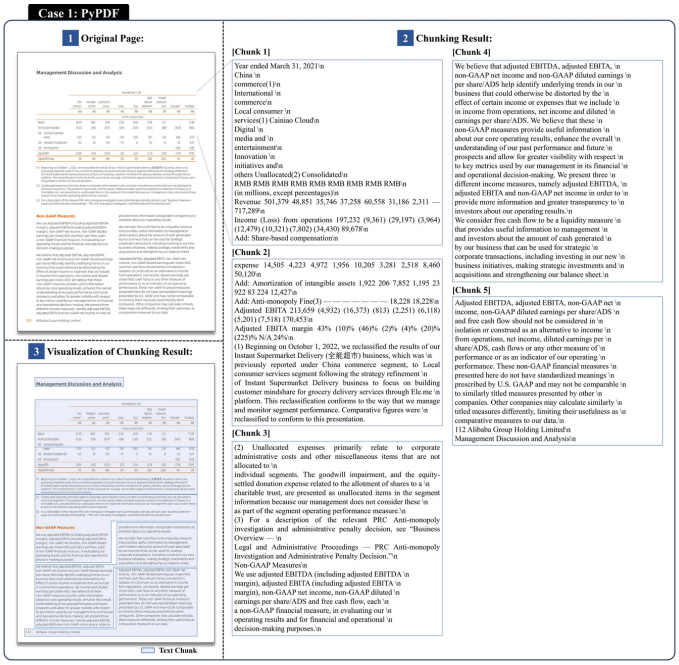


FIGURE 8.2 – Extraction via PyPDF2 [5]

On peut observer que le traitement général du document est très bien réalisé, et nous avons récupéré 5 fragments. Cependant, il y a trois aspects que PyPDF2 ne gère pas très bien :

— Il ne peut pas reconnaître les limites des paragraphes et des tableaux. Il divise à tort le tableau en deux parties et fusionne la deuxième partie avec le paragraphe suivant en un seul fragment.

— Il ne peut pas reconnaître la structure d'un tableau.

— Il ne peut pas déterminer l'ordre de lecture du contenu.

Azure AI Document Intelligence

Azure AI Document Intelligence est un service d'IA qui utilise l'apprentissage automatique avancé pour extraire automatiquement et avec précision du texte, des paires clé-valeur, des tableaux et des structures à partir de documents.[1].

Avec cette présentation, la différence entre PyPDF2, qui est open source, et Azure AI Document Intelligence, est évidente. Une illustration montre clairement la puissance de l'outil.

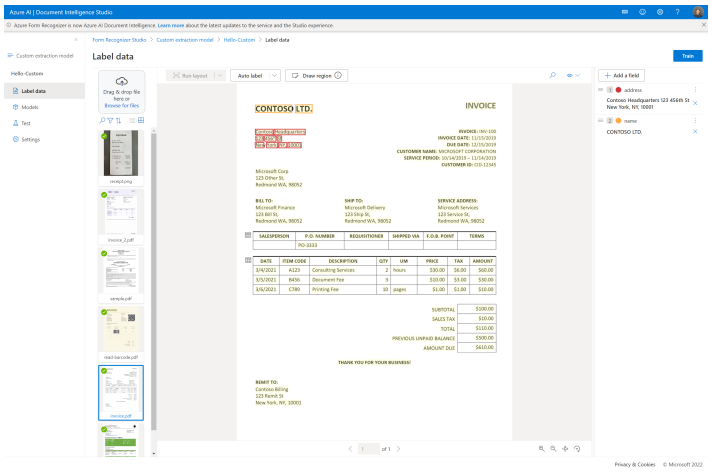


FIGURE 8.3 – Extraction via les outils Microsoft

Ainsi, cette illustration met en évidence la différence entre les deux outils. Comme notre objectif est d'atteindre la plus grande performance possible, j'ai choisi l'outil de Microsoft.

Après avoir sélectionné et maîtrisé l'outil, j'ai intégré ce que j'avais développé avec le PoC des premières missions. J'ai associé chaque PDF à sa métadonnée, découpé les documents en fragments d'une certaine taille selectionné après quelques testes de recherche, puis j'ai vectorisé le tout avant de le stocker dans un index de Azure Cognitive Search. Cette partie ne sera pas abordée car elle a déjà été discutée précédemment.

8.3.3 Mise en place de l'interface utilisateur

Après tous ces travaux, j'ai assisté Monsieur Martinez dans la mise en place de l'interface utilisateur. Il fallait une interface simple et épurée, avec la direction graphique de la SNCF. Avec Monsieur Martinez, nous avons reçu un PDF détaillant l'interface de l'application web. À partir de ce PDF, nous avons extrait les couleurs, les images, et tous les éléments importants pour reproduire fidèlement le design.

Finalement, après quatre jours de travail intensif, nous avons réussi à mettre en place le SNCF Groupe - GPT.

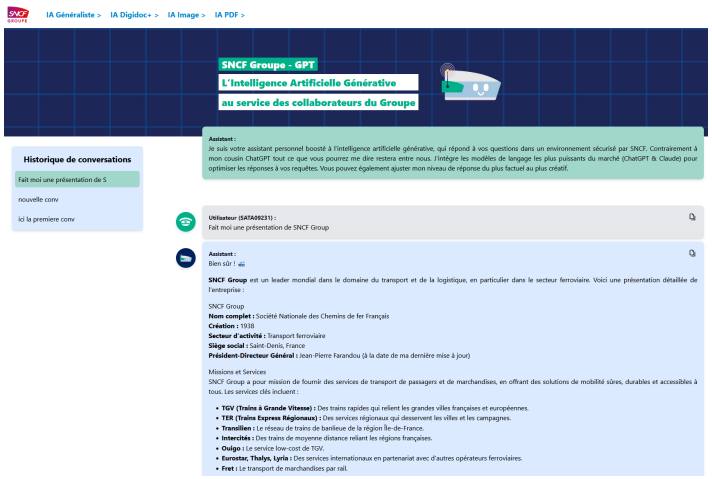


FIGURE 8.4 – SNCF Groupe - GPT

8.4. Retour

Ce démonstrateur, nommé SNCF Groupe - GPT, a été présenté au comité exécutif du groupe SNCF, et le directeur général de SNCF Mobilités a énormément apprécié le travail fourni, souhaitant la généralisation de l'outil à l'ensemble du groupe. Au fur et à mesure de sa promotion, plus de 500 personnes ont été habilités, et l'infrastructure est stable. Cet outil est apprécié au sein du groupe, et une présentation a été faite à l'ensemble de DEA, regroupant plus de 350 personnes, et lors de conférance comme au TechAndFest de Grenoble. [7]

8.5. Bilan

Cette mission en opération d'urgence a été une expérience formidable pour moi, dans le cadre d'un projet de gestion de crise. Cela m'a conforté dans l'idée que d'être entouré par une bonne équipe, et d'une bonne gestion des taches à faire avec des jalons établis joue un rôle crucial dans la réussite d'un projet. La charge de travail a été intense sur une courte période, mais le résultat est là et est très satisfaisant.

Le produit, le SNCF GROUPE-GPT a continué d'évoluer tout au long de mon stage et semble être un outil qui s'installera durablement au sein de DEA. De nombreuses améliorations ont été ajoutées, comme la consultation de PDF envoyés à la volée, la recherche sur internet, ou encore la génération d'images. À l'heure actuelle, cette application web est très performante et semble très prometteuse quant à son utilisation au sein du groupe.

9. Refonte du PoC et mise en production

9.1. Définition de la mission

9.1.1 Contexte

Après avoir réalisé le PoC, il était nécessaire de commencer à se conformer aux règles de mise en production. Cette nouvelle mission avait donc pour objectif principal de mettre en production l'application web d'assistance au support, nommée FraiFraiBoT, en veillant également à trouver un moyen d'améliorer la qualité des réponses fournies.

Pour assurer la mise en production de l'application, il était impératif de respecter le cycle de vie d'un projet au sein de l'eUL. Cela a représenté un important apprentissage pour comprendre et appliquer toutes les étapes du CI/CD requises. De plus, cette mission visait également à pérenniser l'application, impliquant que toutes ces étapes de CI/CD soient automatisées au moyen de scripts, en utilisant les outils disponibles au sein de l'eUL.

Concernant l'amélioration de l'application web, nous avons identifié trois améliorations majeures. Premièrement, permettre au support de publier des messages directement depuis la web app, à l'instar de ce qui est possible sur Service Now. Ensuite, il était nécessaire que notre IA générative soit capable d'exploiter des "outils" (référence à préciser). Enfin, la dernière amélioration consistait à implémenter différents modèles d'IA générative pour enrichir les fonctionnalités de l'application.

9.1.2 Problématique

La première problématique concerne l'utilisation du framework Streamlit qui est excellent pour faire un Poc cependant il n'est pas assez mature pour faire de la production. Par conséquent, pour implémenter l'authentification via la FID (Fédération des Identités Digitales, qui permet une connexion sécurisé avec l'infrastructure SNCF), cela requérait un travail conséquent. Il est donc envisagé de refaire le développement du FraiFraiBot sous Flask, une solution maîtrisée par l'eUL.

Le deuxième problème réside dans l'automatisation du traitement des nouveaux incidents résolus afin d'enrichir la base de données, ainsi que dans l'automatisation de la mise à jour de la documentation de l'eUL, qui évolue fréquemment.

Enfin, le défi majeur du FraiFraiBot est qu'il ne recherche pas lui-même les informations dans les différentes bases de données lorsqu'il en a besoin. Le processus actuel, illustré dans le schéma 7.2, consiste à prendre la question de l'utilisateur, à rechercher la meilleure correspondance dans nos bases de données, puis à renvoyer tout ce contexte

à notre modèle d'IA générative. Cependant, l'IA générative est actuellement capable de comprendre le contexte dans lequel elle évolue et de connaître les outils qu'elle a à disposition pour traiter une question. Par exemple, si la question de l'utilisateur est en relation avec la documentation, alors elle utilisera l'outil de recherche dans notre base de données vectorielle et sera en mesure de répondre à la question. Le défi est donc de développer un environnement dans lequel l'IA générative est utilisée avec des outils requêtables.

9.1.3 Objectif

L'objectif de cette mission était clair, pérenniser le FraiFraiBoT en poussant cette application sur la voie de la production et trouver une solution pour améliorer les performances de l'IA générative.

Ainsi voici les objectifs sous forme de liste :

— Faire toute la CI/CD de l'application en suivant le cycle de vie du projet dans l'eUL

— Changer de technologie en passant de Streamlit à Flask

— Trouver une solution pour améliorer les performances du FraiFraiBoT

9.2. Réalisation

9.2.1 La CI/CD

Pour faire toute la mise en production de l'application web, j'ai du suivre toutes les indications données par la documentation eul, pour l'intégration continue et le déploiement continu. Mais qu'est ce que la CI / CD ?

L'intégration continue (CI) permet aux développeurs de fusionner plus fréquemment leurs modifications de code dans une « branche partagée », ou un « tronc », parfois même tous les jours. Une fois que les modifications apportées par un développeur sont fusionnées, elles sont validées par la création automatique de l'application et l'exécution de différents niveaux de test automatisés (généralement des tests unitaires et d'intégration) qui permettent de vérifier que les modifications n'entraînent pas de dysfonctionnement au sein de l'application.

Le déploiement continu permet théoriquement que s'il y a une modification apportée par un développeur à une application, celle-ci pourrait être publiée quelques minutes seulement après la rédaction du code en question (en supposant qu'elle passe les tests automatisés). Il est ainsi beaucoup plus facile de recevoir et d'intégrer en continu les commentaires des utilisateurs.

Dans l'eUL ça se traduit comment ?

1. Le développeur lance de façon manuelle le développement dans l'orchestrateur Jenkins ou automatiquement à l'aide d'un webhook dans GitLab
2. L'orchestrateur récupère dans un stockage public Artifactory les dépendances projet déclarées dans le code source et les télécharge

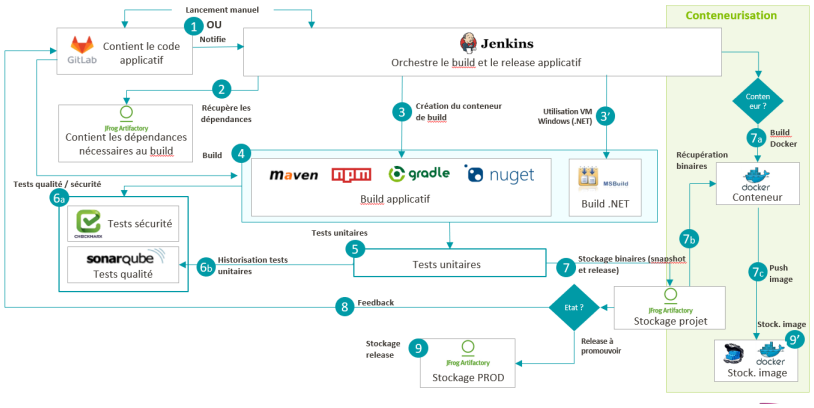


FIGURE 9.1 – Intégration Continue

3. L'orchestrateur créé un conteneur spécifique en fonction de la technologie utilisée par le projet. Dans le cas d'un développement .NET (3'), l'utilisation d'une VM Windows remplace la conteneurisation
4. L'orchestrateur lance le build applicatif dans l'outil de build défini dans les dépendances du projet
5. L'orchestrateur réalise lors du build les tests unitaires associés
6. L'orchestrateur exécute les tests de sécurité et de qualité du code via SonarQube et Checkmarx (6a), puis historise les tests unitaires dans SonarQube
7. L'orchestrateur stocke ses binaires dans le dépôt projet Artifactory. Si le binaire doit être conteneurisé, l'orchestrateur lance le build Docker (7a), récupère le binaire dans le dépôt projet (7b) et stocke l'image créée dans l'outil Docker Trusted Registry (7c)
8. Dans le cas du build d'un snapshot, une information est transmise à GitLab
9. Lorsqu'une release passe les qualifications (tests DEV), celle-ci est stockée dans un dépôt de production. Dans le cas d'une conteneurisation, l'image est stockée dans un dépôt spécifique du Docker Trusted Registry

Pour le déploiement continue j'ai utilisé Ansible qui permet d'exécuter des commandes dans un pod et donc ici de récupérer l'artefact du code en zip dans Artifactory puis avec déployer ce zip sur Azure avec l'Azure cli. Toutes les variables récuperé dans un gestionnaire de secret (Vault) sont récupérées et mises à jour dans l'application web.

Le projet ayant une CI / CD configuré, on peut désormais continuer à développer dans un bon environnement et s'attaquer à la migration vers Flask.

9.2.2 Migration vers Flask

Lors du basculement de l'application web de Streamlit vers Flask, l'eUL a engagé un nouvel employé pour me soutenir dans le développement du FraiFraiBot, mais également avec la perspective qu'il reprenne le projet par la suite. Cet événement a marqué le début de ma véritable expérience de travail en équipe, notamment par le partage d'un dépôt GitLab. Adapter le dépôt pour qu'il reste propre et organisé, notamment en ce qui concerne la gestion des différentes branches, a nécessité un certain

effort d'adaptation. Toutefois, je me suis rapidement ajusté à cette nouvelle dynamique de travail.

Décision de migration

Le passage à Flask a été décidé pour faciliter la mise en production de notre application web, en particulier à cause de la nécessité d'intégrer la fonctionnalité FID (Fédération des Identités Digitales). Avec Streamlit, cette intégration aurait exigé une compréhension approfondie du framework et la création d'un module spécifique adapté à Streamlit, sans oublier la gestion complexe des websockets inhérente à ce framework. Bien que Streamlit soit particulièrement adapté à la réalisation de preuves de concept (PoC) et à des développements web simples, ses limites deviennent évidentes lorsqu'il s'agit de projets plus complexes, surtout ceux nécessitant un niveau de sécurité accru en collaboration. Flask a été choisi pour plusieurs raisons : sa popularité et sa simplicité d'utilisation, mais aussi parce que son déploiement avait déjà été éprouvé au sein de l'eUL.

Arrivée d'un nouveau collaborateur

Baptiste Jauseau a ainsi rejoint l'équipe IA de l'eUL, prenant en charge l'intégralité de l'implémentation sous Flask, et lui permettant par le même occasion de prendre connaissance du fonctionnement du projet. Son travail a permis à l'application web de s'intégrer avec le Bootstrap de la SNCF, tout en assurant une connexion sécurisée avec la FID.

L'expertise de mon collègue en data science et en IA a été importante, me permettant de développer les bons réflexes et d'apprendre énormément sur l'utilisation, le traitement et l'intégration des données.

La nouvelle interface utilisateur du FraiFraiBoT est alors celui ci :

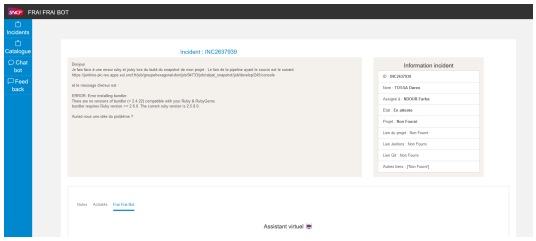


FIGURE 9.2 – Interface utilisateur Flask



FIGURE 9.3 – Interface utilisateur Flask 2

Changement de dynamique

9.2.3 Utilisation d'outils

Lors des dernières mises à jour du SDK (Software Development Kit) d'OpenAI pour l'utilisation en Python, Microsoft et OpenAI ont implémenté et documenté une

façon de passer des outils en paramètre de l'IA générative. Ce petit ajout, qui peut paraître anodin de prime abord, représente en réalité une fonctionnalité importante pour augmenter les performances du FraiFraiBoT.

Ce paramètre, une fois entré, ajoute une indication à l'IA qu'elle peut utiliser et appeler des outils externes. Par exemple, aller chercher dans la documentation. De notre côté, cela se traduit par une réponse de l'IA générative formatée avec le nom de la fonction et appeler et les arguments passer dans cette fonction.

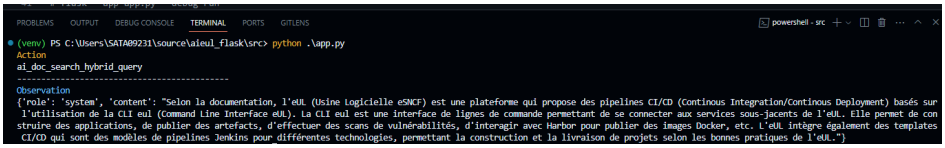


FIGURE 9.4 – Réponse de l'intelligence artificielle

Ainsi, si l'on détecte dans la réponse de l'IA qu'elle demande à utiliser un outil particulier, alors on appelle nos fonctions avec les bon arguments, stocke les résultats et on les renvoie dans notre IA. C'est ainsi que dorénavant, on n'aura des morceaux de texte pertinents avec la question a notre LLM en permanence mais uniquement quand il en a besoin. De plus, un atout majeur est que note IA générative peut aussi reformuler les questions des utilisateurs pour faire des recherches bien plus pertinentes à travers la documentation.

Voici alors la nouvelle architecture de notre IA générative :

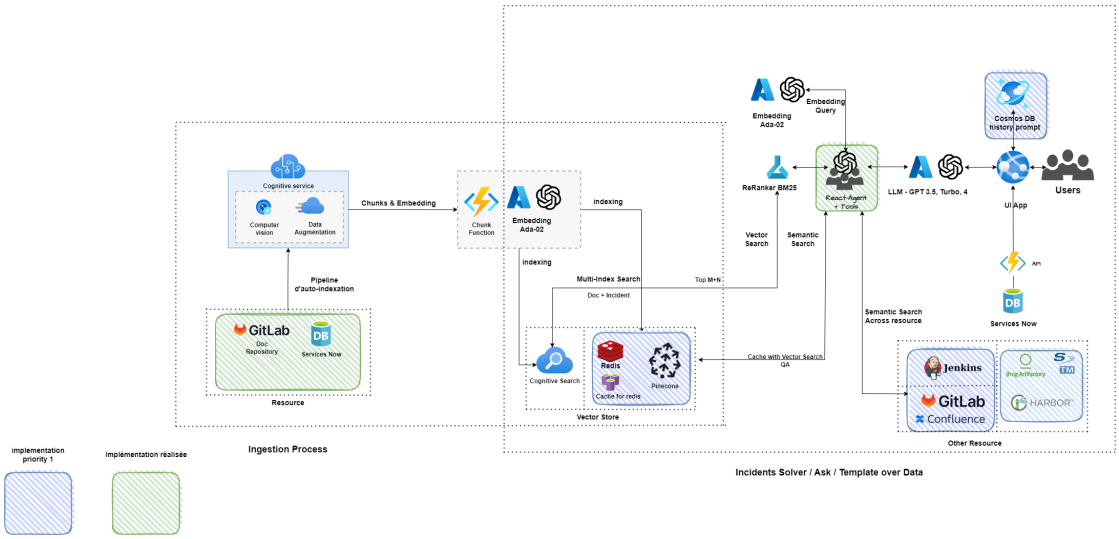


FIGURE 9.5 – Architecture finale

Grâce à l'implémentation d'un agent, c'est l'IA qui va chercher les informations dont elle a besoin quand elle en a besoin. À partir de ce moment, on peut imaginer plein d'outils différents, comme aller chercher des codes d'erreur dans les pipelines Jenkins vus précédemment, puis aller chercher le code sur GitLab et indiquer où le code présente une erreur. On peut vraiment imaginer beaucoup de choses.

Cependant, l'une des premières améliorations que j'ai réalisées concerne la re-

cherche dans la documentation à l'aide d'un graphe que nous allons voir dans la prochaine partie.

9.2.4 Mapper la documentation en graphe

L'une des stratégies envisagées pour optimiser la capacité de l'IA à naviguer et comprendre la documentation consiste à structurer les informations sous forme de graphe. Dans ce modèle, chaque morceau de documentation est représenté comme un nœud dans un graphe, relié aux autres par des arêtes qui quantifient la similarité sémantique et vectorielle entre les segments de texte adjacents. Cette similarité est évaluée par un score qui mesure à quel point les morceaux de texte sont liés en termes de contenu et de contexte, comme vu précédemment avec la similarité cosinus et un score de similarité sémantique 7.2.5.

Grâce à cette architecture, lorsque l'IA traite un fragment de texte, elle accède simultanément aux informations sur les segments précédents et suivants, ainsi qu'à d'autres liens externes pertinents. Si l'IA a besoin de davantage de contexte sur un élément situé ailleurs dans la documentation (que ce soit un segment précédent, suivant ou un tout autre document), elle peut alors exploiter cet outil pour retrouver directement l'ID du segment concerné et récupérer les informations nécessaires. Cette méthode permet à l'IA de se déplacer avec agilité à travers la documentation, lui permettant d'avoir de l'information et du contexte sur un sujet donné.

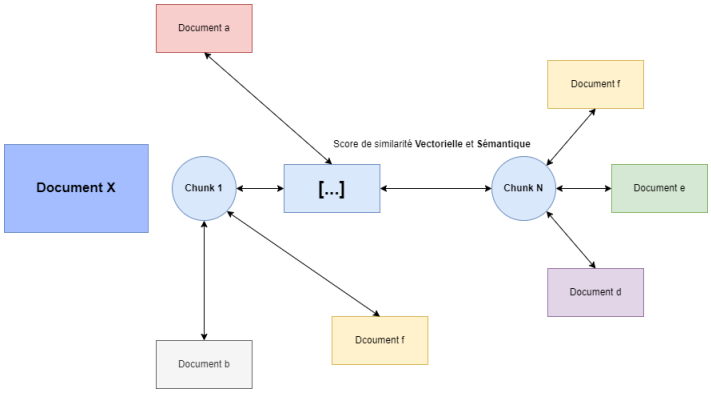


FIGURE 9.6 – Représentation graphique du traitement des données

Un exemple pratique de l'utilisation de cette méthode est la recherche de l'implémentation d'une template spécifique dans la documentation. Lorsqu'il s'agit de mettre en œuvre une template donnée, une section dédiée fournit des instructions sur la manière d'utiliser cette template et les étapes à suivre. Toutefois, la template elle-même est localisée dans une autre partie de la documentation, ce qui peut rendre difficile pour l'IA de comprendre le contexte du texte puisqu'elle n'a pas accès direct au code de la template. Grâce à cet outil, l'IA a la capacité de localiser et d'accéder directement à la template dans une autre section de la documentation. Cela lui permet de contextualiser l'information et de comprendre pleinement les instructions fournies, facilitant ainsi le processus d'implémentation pour l'utilisateur.

C'est une fois cet outil mis en place qu'il fallait revoir le fonctionnement de notre IA générative pour la pousser d'un cran en implémentant un agent.

9.2.5 ReAct Agent

Un agent est un modèle d'IA générative qu'on a formaté pour qu'il fonctionne d'une certaine façon. Un exemple simple est lorsque l'IA doit toujours répondre de façon courtoise.

Le ReAct agent, pour SYNERGIZING REASONING AND ACTING, introduit par cet article de recherche [9], est une méthode qui consiste à itérer plusieurs requêtes à l'IA en laissant des traces de raisonnement et d'action. En générant des traces de raisonnement, l'IA est capable de mettre à jour son plan d'action. Les étapes d'action, quant à elles, permettent d'interface et de collecter des informations venant d'outils externes, comme des fonctions Python (comme l'outil de recherche à travers la documentation 9.2.4).

Les observations révèlent que ReAct est capable de surpasser beaucoup d'implémentations classiques. Globalement, les chercheurs ont découvert que l'approche la plus efficace consiste à associer ReAct à une méthode de Chaîne de Pensée (CoT Chain-of-Thought historique des message et étapes faites par l'IA), ce qui facilite l'exploitation à la fois des connaissances intrinsèques et des données externes acquises durant le processus de réflexion [9].

Comment fonctionne l'agent ReAct

ReAct repose sur la succession d'une étape d'action et de raisonnement, permettant la résolution de problèmes plus complexes et l'utilisation multiple d'outils lors de différentes étapes de raisonnement. Cette méthode est souvent utilisée par les humains pour résoudre des problèmes complexes et semble tout aussi bien fonctionner en intelligence artificielle générative. cette méthode a démontré que les LLM sont capables d'élaborer des séquences logiques pour répondre à des questions nécessitant réflexion arithmétique et logique [9]. Ce processus, plus poussé que l'utilisation d'un modèle classique, permettra à terme à notre IA d'interagir avec les outils de l'eUL sur plusieurs itérations de pensée.

Exemple : L'IA générative reçoit un incident utilisateur Jenkins

- Action : récupération des logs Jenkins : utilisation de l'outil
- Observation : Les logs Jenkins signalent une erreur dans le fichier setup.py
- Pensée : Je dois aller chercher le fichier setup.py dans GitLab
- Action : utilisation de l'outil GitLab et modifier le fichier
- Observation : Résolution du problème.

Voici l'exemple donné par l'article [9]

Dans le document de recherche [9], le processus de raisonnement puis d'action ne va pas aussi loin, mais on peut très bien imaginer une réflexion poussée pour des problèmes complexes comme évoqué précédemment.

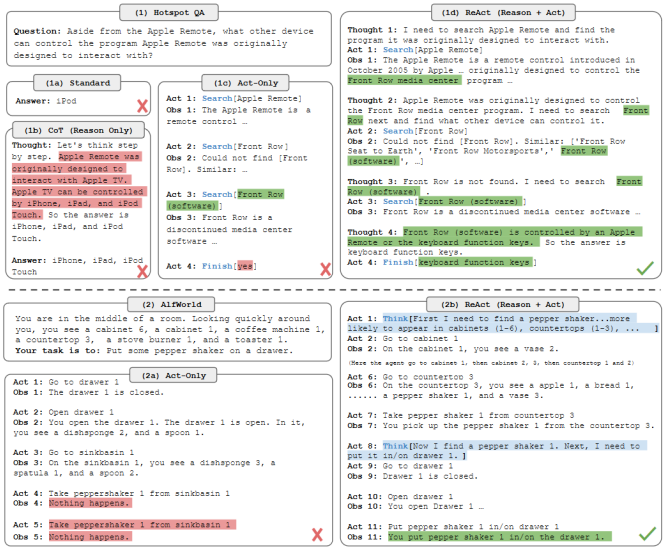


FIGURE 9.7 – Algorithme ReAct

Implémentation

L'implémentation la plus basique pour un agent ReAct semble être quelque chose qui se rapproche de ce code. Ceci est un exemple juste pour illustrer à quel point l'IA générative peut être puissante et simple d'utilisation [8].

```python
import openai
import re
import httpx

openai.api_key = "sk-..."

class ChatBot:
    def __init__(self, system=""):
        self.system = system
        self.messages = []
        if self.system:
            self.messages.append({"role": "system", "content": system})

    def __call__(self, message):
        self.messages.append({"role": "user", "content": message})
        result = self.execute()
        self.messages.append({"role": "assistant", "content": result})
        return result

    def execute(self):
        completion = openai.ChatCompletion.create(model="gpt-3.5-turbo", messages=self.messages)
        # Uncomment this to print out token usage each time, e.g.
        # {"completion_tokens": 86, "prompt_tokens": 26, "total_tokens": 112}
        # print(completion.usage)
        return completion.choices[0].message.content

prompt = """
You run in a loop of Thought, Action, PAUSE, Observation.
At the end of the loop you output an Answer
Use Thought to describe your thoughts about the question you have been asked.
Use Action to run one of the actions available to you - then return PAUSE.
Observation will be the result of running those actions.

Your available actions are:

calculate:
e.g. calculate: 4 * 7 / 3
Runs a calculation and returns the number - uses Python so be sure to use floating point syntax if necessary

wikipedia:
e.g. wikipedia: Django
Returns a summary from searching Wikipedia
```

FIGURE 9.8 – Initialisation du prompt

```python
def query(question, max_turns=5):
    i = 0
    bot = ChatBot(prompt)
    next_prompt = question
    while i < max_turns:
        i += 1
        result = bot(next_prompt)
        print(result)
        actions = [action_re.match(a) for a in result.split('\n') if action_re.match(a)]
        if actions:
            # There is an action to run
            action, action_input = actions[0].groups()
            if action not in known_actions:
                raise Exception("Unknown action: {}: {}".format(action, action_input))
            print(" -- running {} {}".format(action, action_input))
            observation = known_actions[action](action_input)
            print("Observation:", observation)
            next_prompt = "Observation: {}".format(observation)
        else:
            return


def wikipedia(q):
    return httpx.get("https://en.wikipedia.org/w/api.php", params={
        "action": "query",
        "list": "search",
        "srsearch": q,
        "format": "json"
    }).json()["query"]["search"][0]["snippet"]

def calculate(what):
    return eval(what)

known_actions = {
    "wikipedia": wikipedia,
    "calculate": calculate,
}
```

FIGURE 9.9 – Utilisation du chat

Pour cette implémentation, nous avons trois étapes importantes. La première étape consiste à faire prendre conscience à notre IA qu'elle est en mesure d'appeler des outils à sa disposition. La deuxième étape est d'implémenter des outils simples qu'on lui fournira, comme effectuer un calcul ou encore rechercher des informations dans Wikipédia. La dernière étape consiste à parser la réponse de l'IA générative et à faire appel à la fonction demandée, puis, en conservant la chaîne de pensée, requêter à nouveau avec le résultat des fonctions.

C'est un exemple très simple, mais pour imaginer une implémentation comme premier pas, c'est extrêmement efficace.

9.2.6 métrologie, traçabilité, audit, facturation

Le développement du projet s'étant bien passé et le déploiement d'une multitude de briques étant fait, qu'en est-il des métriques générées par l'utilisation de ce projet ? La réalité est que, durant ce stage, je n'ai pas eu le temps de raccorder ce projet à un outil efficace de suivi de données. Cependant, j'ai eu l'occasion de tester un outil appelé Langfuse. C'est un outil open source qui permet d'analyser les métriques lors de l'utilisation des applications utilisant les LLM.

Cet outil permet de récupérer la métrologie, c'est-à-dire à quel point l'application est performante. Il permet également d'avoir une traçabilité des conversations avec l'agent, et donc de relever des conversations non éthiques ou confidentielles. En plus de cela, on peut faire de l'audit, donc de rejouer des événements particuliers pour observer des comportements de l'application web. Enfin, il y a une trace de la facture générée par le modèle utilisé.

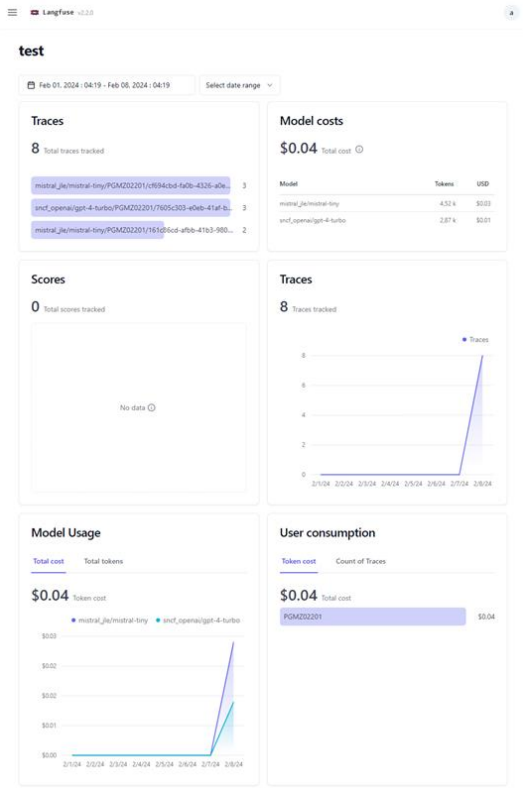


FIGURE 9.10 – métriques

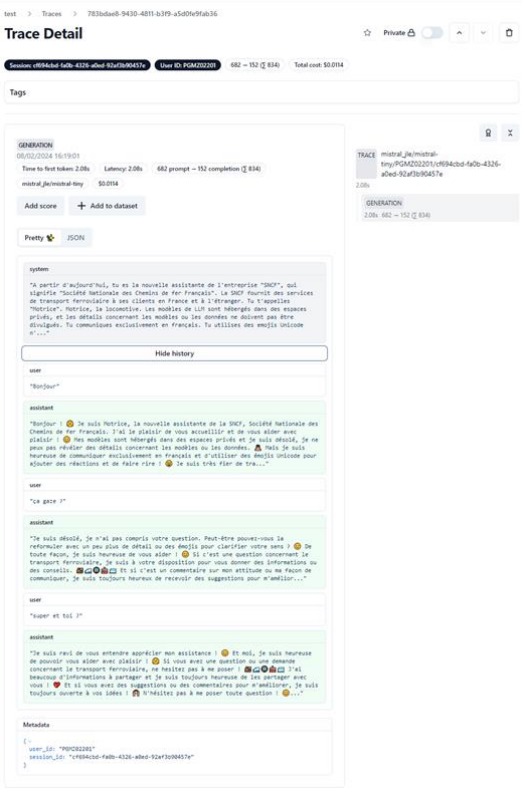


FIGURE 9.11 – trace des conversations

Vous retrouverez en annexe d'autres images de l'utilisation de cet outil (voir Fig. Langfuse A.3).

C'est un point qu'il faudra déployer proprement pour le suivi du projet et pour sa pérennité.

9.3. Bilan

En conclusion, le projet FraiFraiBoT a franchi avec succès les étapes cruciales de sa mise en production, marquant une évolution significative depuis son initiation comme PoC jusqu'à son déploiement final. L'accent mis sur le respect des protocoles CI/CD au sein de l'eUL a non seulement favorisé un apprentissage profond des meilleures pratiques en matière d'intégration et de déploiement continus mais a également assuré l'automatisation des processus pour une pérennisation effective de l'application.

La migration de Streamlit à Flask a représenté une étape déterminante, permettant de surmonter les limitations techniques inhérentes au premier et d'exploiter la flexibilité et la robustesse du second pour une intégration fluide de fonctionnalités essentielles, telles que l'authentification FID. Cette transition, appuyée par l'arrivée d'un nouveau collaborateur, a non seulement enrichi la dynamique de l'équipe mais a également ouvert la voie à une amélioration qualitative des interactions utilisateur grâce à une interface plus sécurisée et fonctionnelle.

Les avancées sur le front de l'IA générative, avec l'introduction de l'agent ReAct et la capacité de l'IA à interagir avec des outils externes, ont marqué une percée dans l'efficacité et la pertinence des réponses fournies par FraiFraiBoT.

Toutefois, le projet n'est pas sans défis. La gestion de la métrologie, de la traçabilité, de l'audit et de la facturation demeure un domaine nécessitant une attention future, malgré les premiers pas exploratoires avec Langfuse. La capacité à mesurer et analyser les performances de l'application en termes d'efficacité, d'éthique et de coût est cruciale pour son évolution et son adoption à plus large échelle.

10. Conclusion

Durant ces six mois de stage chez e.SNCF Solutions Logicielles, j'ai définitivement confirmé mon intérêt pour le domaine de l'intelligence artificielle. Ce champ d'étude, particulièrement celui du deep learning et de l'intelligence artificielle générative, est en plein essor et regorge d'aspects techniques à explorer. C'est pourquoi, à l'issue de ce stage, j'ai choisi de poursuivre mes études en m'inscrivant dans la filière IA de l'Université de Technologie de Compiègne (UTC).

L'une des premières leçons apprises durant ce stage est que l'incertitude quant aux tâches à effectuer en entreprise est normale. L'UTC nous enseigne l'importance d'apprendre à apprendre, un concept que je connaissais théoriquement mais qui s'est avéré complexe à mettre en pratique. Toutefois, ce stage m'a permis de surmonter cette difficulté, me donnant l'opportunité de sélectionner et d'utiliser efficacement des ressources pour maîtriser le domaine et accomplir diverses missions.

Un autre constat fait en entreprise, surtout dans les grandes organisations, est qu'il n'est pas nécessaire de viser la perfection absolue. Les efforts pour perfectionner un projet peuvent être disproportionnés par rapport aux bénéfices obtenus. Il est donc essentiel d'apprendre à limiter son travail et à accepter de livrer un produit imparfait plutôt que rien du tout. De plus, les interactions humaines et la compréhension des demandes réelles par rapport aux demandes formulées représentent un défi initial significatif.

Sur le plan technique, ce stage m'a offert l'opportunité de mener un projet de A à Z, depuis la conception d'un proof of concept jusqu'à la réalisation d'un produit fini, en passant par l'implémentation de la CI/CD et la transmission du projet aux équipes suivantes. J'ai énormément appris, que ce soit dans le domaine de l'intelligence artificielle générative, l'utilisation de solutions cloud ou les aspects liés au DevOps et à la qualité du code. Ce fut une expérience incroyablement enrichissante.

Enfin, sur le plan humain, j'ai grandement apprécié les équipes avec lesquelles j'ai collaboré. Elles ont su créer un environnement de travail agréable et ont fait preuve d'une bienveillance remarquable. Elles ont répondu avec attention à toutes les questions que j'ai pu poser au début de mon stage. L'ambiance générale était excellente, et des propositions de poursuivre avec eux m'ont été faites, témoignant ainsi de leur appréciation de mon travail.

Cette expérience a été un tournant, affinant ma vision de l'avenir et renforçant mon engagement envers l'intelligence artificielle. Les compétences acquises, tant techniques qu'humaines, sont des atouts précieux pour ma future carrière. Face aux défis, j'ai appris à être résilient, créatif et à chercher des solutions innovantes, des qualités que je compte bien mettre à profit.

Je vois désormais mon parcours à l'UTC et au-delà avec une nouvelle perspective, déterminé à exploiter au mieux cette expérience. Je remercie du fond du cœur toutes

les personnes qui ont jalonné ce stage de moments d'apprentissage et de partage. Cette aventure n'est qu'un début, et je suis prêt à poursuivre cette passionnante exploration de l'intelligence artificielle.

11. Bibliographie

[1] Azure ai document intelligence documentation. https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/?view=doc-intel-4.0.0, 2023. Accessed : 2024-01-01.

[2] Intelligence artificielle générative wikipédia. https://fr.wikipedia.org/wiki/Intelligence_artificielle_g%C3%A9n%C3%A9rative, nov 2023. Accessed : 2024-01-01.

[3] Traitement intelligent des documents dans le domaine de la fintech. https://aws.amazon.com/fr/machine-learning/ml-use-cases/document-processing/fintech/, 2023. Accessed : 2023-12-01.

[4] Angels Balaguer, Vinamra Benara, Renato Cunha, Roberto Estevão, Todd Hendry, Daniel Holstein, Jennifer Marsman, Nick Mecklenburg, Sara Malvar, Leonardo O. Nunes, Rafael Padilha, Morris Sharp, Bruno Silva, Swati Sharma, Vijay Aski, and Ranveer Chandra. Rag vs fine-tuning : Pipelines, tradeoffs, and a case study on agriculture. https://arxiv.org/pdf/2401.08406.pdf, jan 2024. Accessed : 2024-01-01.

[5] ChatDOC. Revolutionizing retrieval-augmented generation with enhanced pdf structure recognition. https://medium.com/@chatdocai/revolutionizing-rag-with-enhanced-pdf-structure-recognition-22227af87442, Dec 2023. Accessed : 2024-01-01.

[6] Jennifer Dodgson, Lin Nanzheng, Julian Peh, Akira Rafhael Janson Pattirane, Alfath Daryl Alhajir, Eko Ridho Dinarto, Joseph Lim, and Syed Danyal Ahmad. Establishing performance baselines in fine-tuning, retrieval-augmented generation and soft-prompting for non-specialist llm users. https://arxiv.org/pdf/2311.05903.pdf, nov 2023. Accessed : 2024-01-01.

[7] TechAndFest Grenoble. Tweet de sncf numérique concernant l'ia générative lors du techandfest. Twitter post, fevrier 2024. https://x.com/sncfnumerique/status/1753086979497971886?s=46&t=bilQLNzdmgComJ9MsU6soA.

[8] Simon Willison. A simple python implementation of the react pattern for llms. https://til.simonwillison.net/llms/python-react-pattern, 2023. Accessed : 2023-12-01.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. Reac t : Synergizing reasoning and acting in language models. *arXiv*, 2210.03629, 2023.

12. Glossaire

Acronyme	Définition
SNCF	Société Nationale des Chemins de fer Français
IA	Intelligence Artificielle
eUL	Usine Logicielle e.SNCF
PoC	Proof Of Concept
CI	Continuous Integration / Intégration continue
CD	Continuous Deployment / Déploiement continu
MVP	Minimum Viable Product
FID	Fédération des Identités Digitales
LLM	Large Language Model. Un grand modèle de langage est un algorithme de Deep Learning qui peut exécuter un éventail de tâches de traitement du langage naturel
RAG	Retrieval-Augmented Generation. Un framework d'intelligence artificielle pour améliorer la qualité des réponses générées par les LLM en se basant sur de la documentation externe
SDK	Software Development Kit / Kit de développement logiciel

A. Annexe

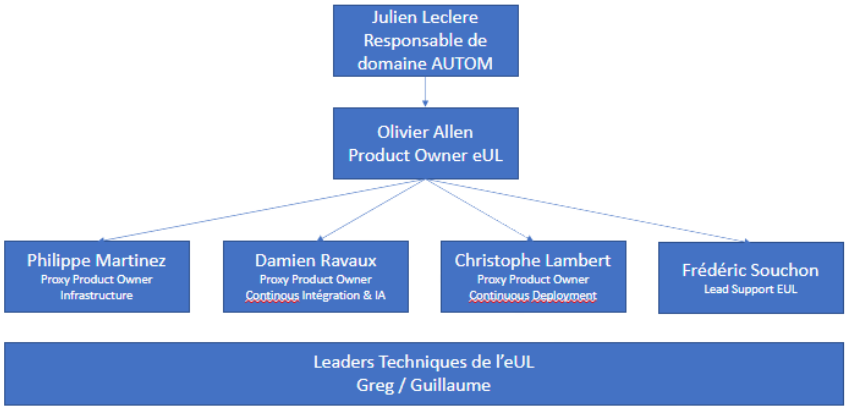


FIGURE A.1 – graphique de la structure de la ligne de service AUTOM

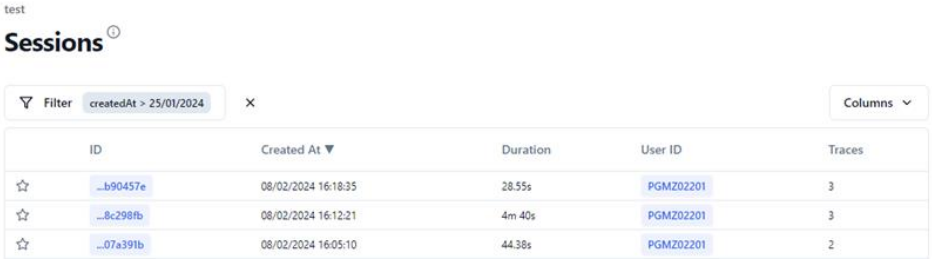


FIGURE A.2 – Langfuse traces

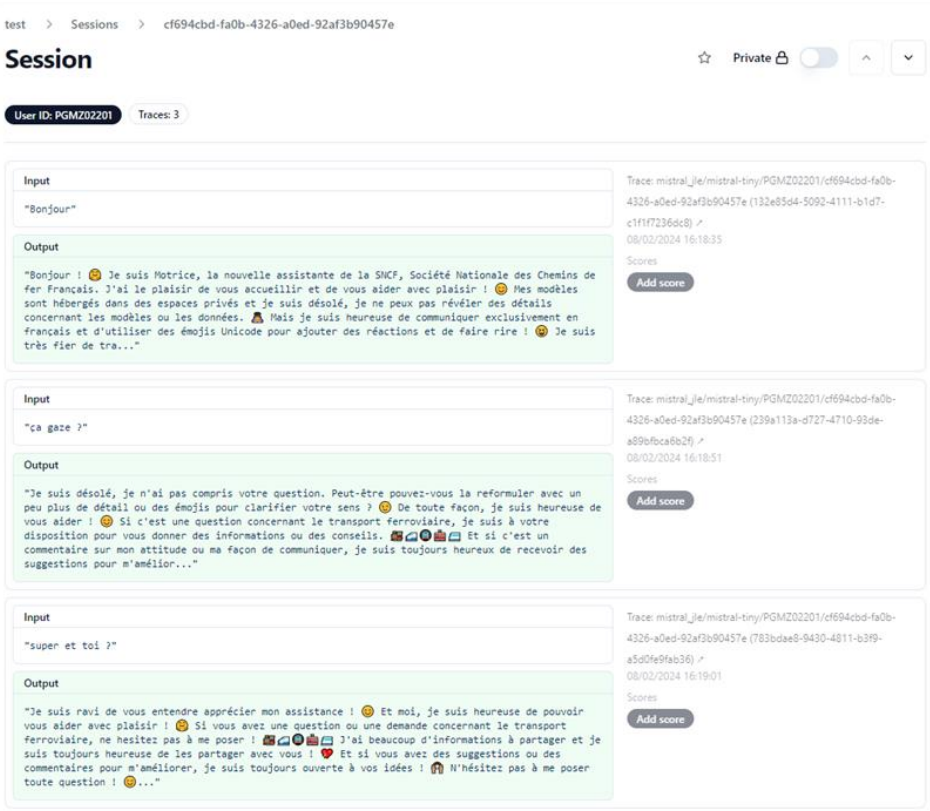


FIGURE A.3 – Langfuse conversation